SMART INVENTORY AND EXPENSE TRACKER WITH AI REPORT FOR KEY PERFORMANCE INDICATORS

Session (2022-2026)

Program

Bachelor of Computer Science

Submitted by:

Muhammad Usman COMSC-F22-020

Supervised by:

Syed Waqar Ali Shah

Lecturer, Department of Computer Science

DEPARTMENT OF COMPUTER SCIENCE

GOVT POSTGRADUATE COLLEGE MANSEHRA, HED, KP

SMART INVENTORY AND EXPENSE TRACKER WITH AI REPORT FOR KEY PERFORMANCE INDICATORS



A report submitted to

Department of Computer Science

Gove Postgraduate College Mansehra, HED, KP

As partial fulfillment of requirements

For award of degree of

Bachelor of Computer Science

DEPARTMENT OF COMPUTER SCIENCE

GOVT POSTGRADUATE COLLEGE MANSEHRA, HED, KP

FINAL APPROVAL

This is to certify that we have read the thesis titled "SMART INVENTORY AND EXPENSE TRACKER WITH AI REPORT FOR KEY PERFORMANCE INDICATORS " submitted by the following students of Bachelor of Studies in Computer Science (2022-2026) of Govt. Postgraduate College, Mansehra.

STUDENT NAME	REGISTRATION NO.
MUHAMMAD USMAN	**COMSC-F22-020**

We judged that this thesis is of sufficient standard to warrant its acceptance by the Department of Computer Science Govt Postgraduate College, Mansehra for the award of Bachelor of Studies (BSCS) degree in Computer Science from Hazara University, Mansehra.

External Examiner Signature:	
External Examiner Name:	Dr. Syed Hamad Shirazi
Designation & Institute:	Assistant Professor Computer Science, Hazara University Mansehra
Internal Examiner Signature:	
Internal Examiner Name:	Mr. Junaid Hussain
Designation & Institute:	Lecturer Computer Science, GPGC Mansehra
Supervisor Signature:	
Supervisor Name:	Syed Waqar Ali Shah
Designation & Institute:	Lecturer Computer Science, GPGC Mansehra
Head of Department Signature:	
Head of Department Name:	Mr. Muhammad Abid
Designation & Institute:	Assistant Professor Computer Science, GPGC Mansehra

DEDICATION

I dedicate this thesis to my parents, whose prayers, sacrifices, and constant support have been the foundation of my academic journey. Their encouragement gave me the strength to remain focused and determined throughout my studies.

I also dedicate this work to my teachers and mentors, whose guidance and knowledge played a vital role in shaping my understanding and skills. Finally, this thesis is dedicated to myself, for the patience, perseverance, and commitment shown in completing this work.

DECLARATION

We hereby declare that the project titled "Smart Inventory and Expense Tracker With AI Reports For Key Performance Indicators" and the accompanying thesis report have been independently developed by me and myself, as a partial requirement for the degree of BSCS. I affirm that this work is the result of our personal efforts and has been carried out under the sincere guidance of our project supervisor Mr. Waqar Shah. No part of this software or report has been copied from any external source, nor does it reproduce the work of any other individual, group, or institution. If any portion of this software or report is ever proven to be copied or plagiarized, I fully accept and stand by all academic and disciplinary consequences. I further confirm that this thesis has not been previously submitted, either jointly or individually, in whole or in part, for any degree or qualification at this or any other academic institution. Additionally, I understand and acknowledge that upon submission, all material related to this project including software, documents, reports, and records become the property of Government Postgraduate College Mansehra, as the part of the official requirements for our BSCS degree.

Name: Muhammad Usman

Signature: _____

ACKNOWLEDGEMENT

I express my profound gratitude to Almighty Allah, whose countless blessings, mercy, and guidance enabled me to complete this Final Year Project titled **Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators** successfully. Without His support, strength, and wisdom, this achievement would not have been possible. I extend my deepest appreciation to my beloved parents, whose unconditional love, prayers, and constant encouragement have been the driving force throughout my academic journey. Their sacrifices and support have always inspired me to work harder and pursue excellence. I am sincerely thankful to my respected supervisor, **Syed Waqar Ali Shah**, for his invaluable guidance, constructive feedback, and continuous support throughout the development of this project. His dedication, expert suggestions, and encouragement have played a pivotal role in shaping my work and enhancing my understanding. I also wish to express heartfelt gratitude to **Sir Muhammad Abid**, whose exceptional teaching, motivation, and strong foundational training have significantly contributed to my learning and technical growth. His support has always been a source of confidence for me. Lastly, I am grateful to all teachers, friends, and classmates who directly or indirectly supported and encouraged me throughout this project.

Muhammad Usman (COMSC-F22-020)

PROJECT IN BRIEF

Project Title	Smart Inventory and Expense Tracker With AI Reports For KPIs
Organization	Government Postgraduate College Mansehra
Undertaken By	Muhammad Usman
Supervised By	Syed Waqar Ali Shah
Starting Month	September 2025
Ending Month	June 2026
Software Used	Android Studio, VScode

PREFACE

This thesis documents the design, development, and evaluation of the Smart Inventory and Expense Tracker with AI Reports for Key Performance Indicators, submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science at Government Postgraduate College Mansehra, affiliated with Hazara University.

The motivation behind this project stems from a direct observation of how small businesses in our local context continue to struggle with manual record-keeping, fragmented data, and the complete absence of data-driven decision-making tools. Watching business owners manage their stock and finances through handwritten notes and basic spreadsheets — while knowing that modern technology could serve them far better — made this project feel meaningful from the very beginning.

The project was undertaken individually over the course of two semesters, from September 2025 to June 2026. It covers the full software development lifecycle: requirements analysis, system design, implementation, testing, and evaluation. The system was built using Flutter for the mobile frontend and Firebase for the cloud backend, with an AI analytics module implemented entirely within the application to deliver expense forecasting and inventory insights without relying on external services.

This thesis is written to be accessible to readers with a general background in computer science. Technical decisions are explained with sufficient context so that the reasoning behind each choice is clear, not just the choice itself. It is hoped that this work serves not only as an academic submission but as a useful reference for anyone looking to build similar lightweight AI-assisted business tools for underserved user groups.

Muhammad Usman - COMSC-F22-020

Department of Computer Science

Government Postgraduate College Mansehra

Abstract

Small businesses in developing economies continue to rely heavily on manual methods — paper ledgers, basic spreadsheets, and informal records — for managing their inventory and tracking business expenses. These approaches are error-prone, time-consuming, and incapable of providing the analytical insights necessary for informed decision-making. Existing digital solutions, while powerful, are typically too expensive, too complex, or too feature-heavy for micro and small enterprises with limited technical capacity.

This project presents the design, development, and evaluation of the Smart Inventory and Expense Tracker with AI Reports for Key Performance Indicators, a mobile application built using the Flutter framework and Firebase backend services. The system enables small business owners to manage inventory items in real time, record and categorize business expenses, and receive intelligent analytical reports generated by an embedded AI analytics module. The application supports full Create, Read, Update, and Delete operations for both inventory and expense records, with live data synchronization powered by Cloud Firestore. A role-based access control system distinguishes between standard users and administrators, with the admin panel providing user management and a full system activity audit log.

The AI analytics module, implemented entirely within the Flutter application using the Dart programming language, performs expense trend analysis by category and by month, forecasts next-period expenses using a simple linear regression algorithm, and generates a prioritized low-stock restocking list ranked by urgency. Charts and recommendation text are presented on a dedicated Reports Screen, making AI-generated business intelligence accessible to non-technical users without requiring external APIs or cloud-based machine learning infrastructure.

The system was evaluated through a multi-level testing programme comprising unit testing, integration testing, system testing, user acceptance

testing, performance testing, and security testing. All twelve functional requirements were confirmed as met. User acceptance testing with five representative small business owners produced average satisfaction scores of 4.4 to 4.6 out of 5 across ease of use, clarity of information, and usefulness of AI reports. Forecasting accuracy testing demonstrated errors below 2% for stable and trending datasets. Performance remained within acceptable thresholds under both normal and moderate data loads, with no application crashes recorded across all test scenarios.

The results demonstrate that lightweight AI-driven business intelligence can be delivered affordably and accessibly through a mobile application, addressing a meaningful gap in the tools currently available to small businesses. The system provides a replicable architectural template for similar applications and a foundation for future extensions including advanced forecasting models, barcode scanning, offline support, and multi-user collaboration.

Keywords: inventory management, expense tracking, mobile application, Flutter, Firebase, AI analytics, linear regression, small business, key performance indicators, role-based access control.

CONTENTS

CHAPTER 1 1
INTRODUCTION 1
1.1 Introduction 2
1.2 Background of Study 2
1.3 Problem Statement 3
1.4 Objectives of the Study 4
1.5 Aims of the Project 5
1.6 Scope of the Project 5
1.6.1 In-Scope 5
1.6.2 Out of Scope 5
1.7 Significance of Study 6
1.7.1 Improved inventory control 6
1.7.2 Better financial management 6
1.7.3 Data-driven decision making 6
1.7.4 Increased transparency and accuracy 6
1.7.5 Scalability and adaptability 6
1.7.6 Academic contributions 6
1.8 Project Features 7
1.9 Structure of Thesis 7
CHAPTER 2 8
LITERATURE REVIEW 8
2.1 Introduction 9
2.2 Existing Inventory Management Systems 9
2.2.1 Manual Inventory and Expense Tracking Systems 10
2.2.2 Spreadsheet-Based Management Solutions 10
2.2.3 Commercial Inventory Software Systems 11
2.3 Review of Related Research Studies 11
2.3.1 Inventory Optimization Techniques 11
2.3.2 Expense Tracking Applications 12
2.3.3 AI-Based Business Analytics Systems 12
2.4 Use of Cloud and AI Technologies 13
2.4.1 Firebase and Cloud Databases 13

2.4.2 AI-Based Reporting and Decision Support 13
2.5 Gaps Identified in Existing Systems 14
2.6 Summary of Literature Review 15
CHAPTER 3 16
SYSTEM METHODOLOGY 16
3.1 Introduction 17
3.2 Overall System Architecture 17
3.3 System Modules Overview 18
3.3.1 User Module 18
3.3.2 Inventory Management Module 19
3.3.3 Expense Tracking Module 19
3.3.4 AI Reporting and Analytics Module 20
3.3.5 Admin Module 20
3.4 Functional Requirements 20
3.5 Non-Functional Requirements 21
3.6 Technology Stack 22
3.6.1 Frontend (Flutter) 23
3.6.2 Backend (Firebase) 23
3.6.3 Database Design 23
3.6.4 AI Tools and Techniques 23
3.7 System Workflow 24
3.7.1 Inventory Workflow 24
3.7.2 Expense Recording Workflow 24
3.7.3 AI Report Generation Workflow 25
3.8 Use Case Diagrams 25
3.9 Data Flow Diagrams 26
3.10 Summary 28
CHAPTER 4 30
SYSTEM DESIGN 30
4.1 Introduction 31
4.2 User Interface Design 31
4.2.1 Design Principles 32
4.2.2 Screen Flow and Navigation 32

4.2.3 Key Screen Descriptions....32
4.3 Database Design....35
4.3.1 Users Collection....36
4.3.2 Inventory Collection....36
4.3.3 Expenses Collection....37
4.3.4 AuditLogs Collection....37
4.4 Class Diagram....37
4.4.1 Model Classes....38
4.4.2 Service Classes....38
4.5 Entity-Relationship Diagram....39
4.6 AI Analytics Component Design....42
4.6.1 Data Pipeline....42
4.6.2 Analytical Algorithms....42
4.7 Summary....43
CHAPTER 5....44
IMPLEMENTATION....44
5.1 Introduction....45
5.2 Development Environment and Tools....45
5.3 Project Structure....47
5.4 Implementation of the Authentication Module....47
5.5 Implementation of the Inventory Management Module....48
5.5.1 Inventory Service....48
5.5.2 Inventory List and Detail Screens....49
5.5.3 Item Form Screen....49
5.6 Implementation of the Expense Tracking Module....49
5.7 Implementation of the AI Analytics Module....50
5.7.1 How the Module Works....50
5.7.2 Reports Screen Layout....51
5.8 Implementation of the Admin Module....51
5.9 Firebase Configuration and Security....52
5.10 Dashboard and Navigation Implementation....52
5.11 Implementation Challenges and Solutions....53
5.12 Summary....54

CHAPTER 6 56
TESTING 56
6.1 Introduction 57
6.2 Testing Strategy 57
6.3 Unit Testing 58
6.3.1 AuthService Unit Tests 58
6.3.2 Inventory Service Tests 58
6.3.3 Expense Service Tests 58
6.3.4 AI Analytics Service Tests 59
6.4 Integration Testing 59
6.4.1 Integration Test Cases and Results 59
6.5 System Testing 61
6.5.1 Functional Requirements Coverage 61
6.6 User Acceptance Testing 63
6.6.1 UAT Tasks 63
6.6.2 UAT Results and Observations 63
6.7 Performance Testing 64
6.8 Security Testing 65
6.9 Summary of Defects and Resolutions 65
6.10 Summary 67
CHAPTER 7 68
RESULTS AND DISCUSSION 68
7.1 Introduction 69
7.2 Objective Fulfilment 69
7.2.1 Mobile application for inventory and expense management 69
7.2.2 Integrate a real-time cloud backend using Firebase 70
7.2.3 Implement an AI analytics module for business insights 70
7.2.4 Implement role-based access control and admin functionality 70
7.3 System Functionality Results 71
7.3.1 Authentication Module Results 71
7.3.2 Inventory Management Results 71
7.3.3 Expense Tracking Results 72
7.3.4 Admin Module Results 72

7.4 AI Analytics Results and Discussion 72
7.4.1 Expense Trend Analysis Results 72
7.4.2 Forecasting Accuracy 73
7.4.3 Low-Stock Ranking Results 74
7.4.4 Auto-Generated Recommendation Text 74
7.5 User Evaluation Results 75
7.6 Performance Results Summary 75
7.7 Comparative Discussion 77
7.7.1 Affordability and Accessibility 77
7.7.2 AI-Driven Insights for Non-Technical Users 77
7.7.3 Real-Time Data Synchronization 77
7.7.4 Limitations Relative to Enterprise Systems 78
7.8 Discussion of Key Findings 78
7.9 Summary 79
CHAPTER 8 80
CONCLUSION AND FUTURE WORK 80
8.1 Introduction 81
8.2 Project Summary 81
8.3 Conclusion 82
8.4 Lessons Learned 83
8.4.1 Agile Development for Academic Projects 84
8.4.2 Firebase as a Rapid Development Platform 84
8.4.3 Importance of User Involvement 84
8.4.4 Managing Scope Effectively 85
8.5 Limitations 85
8.6 Future Work 86
8.6.1 Advanced Forecasting Models 86
8.6.2 Barcode Scanning 87
8.6.3 Data Export 87
8.6.4 Offline Support 87
8.6.5 Push Notifications for Low-Stock Alerts 88
8.6.6 Multi-User Collaboration 88
8.6.7 iOS Support and App Store Deployment 88

8.6.8 Machine Learning-Driven Category Suggestions 89
8.7 Closing Remarks 89
References 91

List of Tables

Table 4.1 Model Classes 38
Table 4.2 Service Classes 39
Table 5.1 Technology And Tools 46
Table 5.2 Challenges and their Solutions 54
Table 6.1 Integration Testing Results 60
Table 6.2 Functional Requirements 62
Table 6.3 UAT Results & Observation 64
Table 6.4 Defects and Resolutions 66
Table 7.1 Forecasting Accuracy 73
Table 7.2 Performance Results 76

List of Figures

Figure 3.1 Use Case Diagram 26
Figure 3.2 Data Flow Diagram 28
Figure 4.1 Dashboard Screen 33
Figure 4.2 Inventory Screen 33
Figure 4.3 Add Expense Screen 34
Figure 4.4 Add Item Screen 34
Figure 4.5 AI Report Screen 35
Figure 4.6 Entity-Relationship Diagram 41

CHAPTER 1

INTRODUCTION

1.1 Introduction

In today's competitive business environment, effective inventory and expense management plays a crucial role in ensuring profitability and operational efficiency. However, many small and medium-sized businesses in developing countries, including Pakistan, still rely on manual registers or basic spreadsheets to track stock and expenses. This often leads to inaccurate records, stock shortages, overstocking, and poor financial decision-making. The **"Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators"** aims to address these challenges by providing a centralized, automated, and intelligent management system.

This system is designed as a user-friendly application that enables businesses to record inventory, track expenses, and monitor sales in real time. The application leverages modern technologies to generate AI-driven reports and insights, such as stock performance analysis and expense trends, helping users make informed decisions. The proposed system is intended for small retail shops and service-based businesses, with an initial focus on local markets. By automating routine tasks and presenting meaningful analytics, the system aims to reduce manual effort, improve accuracy, and enhance overall business efficiency.

1.2 Background of Study

Inventory and expense management are fundamental components of any successful business operation. In Pakistan, however, a large number of small and medium-sized businesses still rely on manual methods such as paper registers, notebooks, or basic spreadsheets to manage their stock and financial records. While these traditional practices are familiar and low-cost, they often lead to problems such as inaccurate inventory counts, unrecorded expenses, stock shortages, overstocking, and lack of clarity in profit and loss calculations.

Small retail shops, local businesses, and service providers frequently face operational challenges due to the absence of digitally integrated systems. These

challenges include delayed stock updates, difficulty in tracking fast-moving items, lack of expense categorization, and limited access to meaningful business insights. As a result, business owners often make decisions based on assumptions rather than data, which can negatively impact profitability and growth.

Recent advancements in mobile application development, cloud-based databases, and artificial intelligence have made it possible to automate and simplify business management processes. Modern inventory systems now leverage real-time data storage, analytics, and intelligent reporting to provide actionable insights. Inspired by the effectiveness of such digital solutions, the proposed **Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators** aims to apply these technologies to small-scale businesses by offering an accessible, data-driven, and intelligent platform for managing inventory and expenses efficiently.

1.3 Problem Statement

The existing inventory and expense management practices used by small and medium-sized businesses in Pakistan are largely manual, inefficient, and prone to error. Most businesses rely on paper-based records or basic spreadsheets to track stock levels, sales, and expenses. This approach leads to several critical issues:

- Inaccurate inventory records due to manual data entry.
- Difficulty in tracking real-time stock availability.
- Poor visibility of daily, monthly, and category-wise expenses.
- No automated alerts for low stock or overstock situations.
- Lack of analytical reports for business performance evaluation.
- Time-consuming record maintenance and data inconsistency.

Due to these challenges, business owners often experience financial losses, poor inventory control, and ineffective decision-making. The absence of intelligent

reporting and data-driven insights further limits their ability to identify trends, control expenses, and plan inventory efficiently.

To address these limitations, there is a critical need for a digitized, automated, and intelligent system that integrates inventory tracking, expense management, and AI-driven reporting to support informed and efficient business decision-making.

1.4 Objectives of the Study

The main goal of this project is to develop a smart, user-friendly inventory and expense tracking system that helps small and medium-sized businesses manage their operations efficiently through automation and AI-driven insights.

- To develop a mobile-based application that allows users to manage inventory and record business expenses digitally.
- To design an intuitive interface for adding, updating, and monitoring stock items in real time.
- To implement expense tracking with category-wise classification for better financial clarity.
- To integrate AI-based reporting features for analyzing sales trends, expense patterns, and stock performance.
- To provide automated alerts for low stock levels and inventory restocking requirements.
- To develop a secure backend system for user authentication and data storage.
- To generate visual reports and dashboards using charts and summaries for business insights.
- To enable data-driven decision support through intelligent suggestions and performance indicators.
- To ensure the system is simple, lightweight, and suitable for non-technical users.

1.5 Aims of the Project

The main aim of this project is to develop a mobile-based **Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators** that automates inventory management, records business expenses, and provides intelligent analytical reports to support efficient decision-making for small and medium-sized businesses.

1.6 Scope of the Project

The scope of this project covers the development of a prototype smart inventory and expense management system with the following modules:

1.6.1 In-Scope

- Mobile-based application for inventory and expense management.
- User authentication and secure login system.
- Inventory management module for adding, updating, and tracking stock items.
- Expense recording with category-wise classification.
- AI-based analytical reports for inventory performance and expense trends.
- Visual dashboards and charts for business insights.
- Low-stock alerts and inventory monitoring features.
- Cloud-based database for real-time data storage and synchronization.

1.6.2 Out of Scope

- Integration with external accounting software or POS systems.
- Online payment gateway integration.
- Large-scale deployment across multiple businesses or regions.
- Advanced AI forecasting models requiring large historical datasets.
- Hardware integration such as barcode scanners or IoT devices.

- However, the proposed system is designed to be scalable and can be enhanced in the future to support advanced integrations and extended business functionalities.

1.7 Significance of Study

The *Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators* holds significant value in improving how small businesses and individuals manage inventory and finances. Its importance is outlined below:

1.7.1 Improved inventory control

Automated stock tracking helps reduce errors, stockouts, and overstocking by maintaining accurate, real-time inventory records.

1.7.2 Better financial management

Systematic expense tracking enables users to clearly monitor spending patterns and control unnecessary costs.

1.7.3 Data-driven decision making

AI-generated reports provide insights, trends, and summaries that support informed business and budgeting decisions.

1.7.4 Increased transparency and accuracy

Centralized digital records eliminate manual bookkeeping issues and improve reliability of financial and inventory data.

1.7.5 Scalability and adaptability

The system can be extended to support multiple users, larger inventories, and additional analytics features in the future.

1.7.6 Academic contributions

The system can be extended to support multiple users, larger inventories, and additional analytics features in the future.

1.8 Project Features

- User authentication and profile management.
- Inventory item addition, update, and deletion.
- Real-time stock level monitoring.
- Expense recording with category-wise classification.
- Automated low-stock alerts.
- AI-based analytical reports and KPI generation.
- Visual dashboards with charts and summaries.
- Secure cloud-based data storage.
- Real-time data synchronization using Firebase.
- Admin or management-level monitoring and control features.

1.9 Structure of Thesis

This thesis is organized into following chapters:

- Chapter 1 – Introduction
- Chapter 2 – Literature Review
- Chapter 3 – System Analysis
- Chapter 4 – System Design
- Chapter 5 – Implementation
- Chapter 6 – Testing
- Chapter 7 – Results
- Chapter 8 – Conclusion and Future Work

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The rapid growth of digital technologies has significantly transformed the way businesses manage their operational and financial activities. Inventory management and expense tracking are two critical components of any organization, particularly for small and medium-sized enterprises, where efficient resource utilization directly affects profitability and sustainability. Traditionally, these tasks have been handled through manual record-keeping or basic digital tools, which are often prone to errors, inefficiencies, and lack of real-time insight.

With the advancement of cloud computing and artificial intelligence, modern systems now offer automated data storage, real-time access, and intelligent analysis capabilities. Inventory management systems have evolved from simple stock-counting tools into integrated platforms that support forecasting, optimization, and decision-making. Similarly, expense tracking applications have progressed from static records to dynamic systems capable of analyzing spending patterns and predicting future financial trends.

This chapter presents a review of existing inventory management and expense tracking systems, as well as relevant research studies related to optimization techniques and AI-based analytics. The purpose of this literature review is to examine current approaches, identify their strengths and limitations, and highlight gaps that justify the development of the proposed Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. By analyzing previous work and existing solutions, this chapter establishes a theoretical and technological foundation for the system design and methodology discussed in subsequent chapters.

2.2 Existing Inventory Management Systems

Inventory management systems are designed to record, monitor, and control stock levels to ensure smooth business operations. Over time, these systems have evolved from manual record-keeping methods to fully automated digital

platforms. Existing systems can broadly be classified into manual systems, spreadsheet-based solutions, and commercial inventory management software. Each category offers certain benefits but also presents limitations that restrict efficiency, scalability, and intelligent decision-making.

2.2.1 Manual Inventory and Expense Tracking Systems

Manual inventory management systems rely on handwritten records, registers, or logbooks to track stock levels and expenses. These systems are commonly used by small businesses due to their simplicity and low initial cost. Inventory quantities, purchases, and expenses are recorded manually whenever transactions occur.

Despite their ease of use, manual systems suffer from several critical drawbacks. They are highly prone to human error, data inconsistency, and record loss. Real-time tracking is not possible, and generating summaries or reports requires significant time and effort. As business operations grow, manual systems become inefficient, unreliable, and unsuitable for informed decision-making.

2.2.2 Spreadsheet-Based Management Solutions

Spreadsheet-based systems, such as those created using Microsoft Excel or Google Sheets, represent a step forward from manual methods. These solutions allow businesses to store data digitally, perform basic calculations, and generate simple charts or summaries. They are widely used due to their flexibility, low cost, and familiarity.

However, spreadsheet-based systems have notable limitations. They require manual data entry, increasing the risk of errors and inconsistencies. Collaboration becomes difficult as data grows, and version control issues may arise. Additionally, spreadsheets lack real-time synchronization, automated alerts, and advanced analytical capabilities. As data volume increases, performance and maintainability become major concerns.

2.2.3 Commercial Inventory Software Systems

Commercial inventory management systems provide comprehensive solutions that include automated stock tracking, reporting, user management, and integration with accounting or sales platforms. Examples include enterprise-level inventory software used by medium and large organizations. These systems offer real-time updates, data security, and advanced reporting features.

Despite their strengths, commercial systems often involve high licensing costs, complex setup processes, and limited customization for small-scale businesses. Many solutions focus primarily on inventory management while offering minimal expense tracking or intelligent analytics. AI-driven insights, predictive analysis, and affordability remain limited, creating a gap for cost-effective, intelligent, and scalable systems tailored to smaller organizations.

2.3 Review of Related Research Studies

Research in the areas of inventory management, expense tracking, and business analytics has expanded significantly with the integration of data-driven and artificial intelligence techniques. Numerous studies have focused on improving operational efficiency, reducing costs, and supporting decision-making through automated systems. This section reviews relevant research works related to inventory optimization, expense tracking applications, and AI-based business analytics.

2.3.1 Inventory Optimization Techniques

Several research studies have explored techniques for optimizing inventory levels to minimize costs while meeting demand. Traditional approaches such as Economic Order Quantity (EOQ), Just-In-Time (JIT), and demand forecasting models have been widely discussed in literature. These methods aim to balance holding costs, ordering costs, and shortage risks.

Recent studies have incorporated machine learning algorithms to enhance demand prediction accuracy. Techniques such as linear regression, time-series

analysis, and clustering have been used to identify consumption patterns and predict future stock requirements. While these approaches improve forecasting accuracy, many proposed models are complex and require large datasets, making them less practical for small-scale businesses with limited historical data.

2.3.2 Expense Tracking Applications

Research related to expense tracking applications primarily focuses on improving financial transparency and budget control. Studies highlight the use of digital expense tracking tools to categorize spending, monitor trends, and reduce unnecessary expenditures. Mobile-based expense tracking applications have gained popularity due to their convenience and accessibility.

However, most existing applications emphasize record-keeping rather than intelligent analysis. Research indicates that many systems fail to provide predictive insights or actionable recommendations. The lack of integration between expense data and inventory data further limits the ability to analyze overall business performance.

2.3.3 AI-Based Business Analytics Systems

AI-based business analytics systems leverage machine learning and data analysis techniques to extract meaningful insights from large datasets. Research in this area demonstrates the effectiveness of AI in forecasting sales, detecting anomalies, and supporting strategic decision-making. Commonly used techniques include regression models, classification algorithms, and clustering methods.

Despite their potential, many AI-based analytics systems are designed for large enterprises and require advanced infrastructure and technical expertise. This creates a barrier for small and medium-sized businesses. Literature highlights the need for simplified, scalable, and cost-effective AI solutions that can be integrated into everyday business applications.

2.4 Use of Cloud and AI Technologies

The integration of cloud computing and artificial intelligence has become a key trend in modern information systems, particularly in business management applications. Cloud technologies provide scalable, secure, and real-time data storage, while AI technologies enable intelligent analysis and predictive decision-making. Together, these technologies form the backbone of smart inventory and expense management systems.

2.4.1 Firebase and Cloud Databases

Cloud databases play a crucial role in ensuring data availability, reliability, and synchronization across devices. Firebase, a cloud-based platform, offers real-time database services, authentication, and secure data storage, making it suitable for mobile application development. Research highlights that cloud databases reduce infrastructure costs and eliminate the need for local server maintenance.

Firebase Firestore supports structured data storage with real-time updates, allowing inventory and expense data to be accessed and updated instantly. Studies show that cloud-based inventory systems improve data accuracy, enable remote access, and support scalability as business operations grow. However, many existing systems use cloud storage only for data persistence and do not fully exploit its potential for analytics and intelligent reporting.

2.4.2 AI-Based Reporting and Decision Support

Artificial intelligence has been widely studied for its ability to analyze historical data and generate meaningful insights. AI-based reporting systems use machine learning algorithms to identify patterns, trends, and anomalies within datasets. In inventory management, AI techniques are applied to forecast demand, detect low-stock conditions, and optimize reorder levels. In expense tracking, AI models help analyze spending behavior and predict future expenses.

Research indicates that AI-driven decision support systems significantly enhance managerial decision-making by reducing uncertainty and manual analysis. However, most AI-based systems are complex and resource-intensive, limiting their adoption by small-scale businesses. There is a growing need for lightweight AI solutions that provide practical insights without requiring extensive computational resources or technical expertise.

2.5 Gaps Identified in Existing Systems

The review of existing inventory management systems, expense tracking applications, and related research studies reveals several limitations that hinder effective adoption, particularly for small and medium-sized businesses. While manual and spreadsheet-based systems are simple and low-cost, they lack automation, real-time access, and analytical capabilities. Commercial software solutions, although feature-rich, are often expensive, complex, and not tailored to the needs of smaller organizations.

One major gap identified is the lack of integration between inventory management and expense tracking. Most existing systems handle these components separately, preventing a unified view of operational and financial data. This separation limits meaningful analysis and makes it difficult to derive insights that support strategic decision-making.

Another significant gap is the limited use of artificial intelligence for practical business analytics. Many systems focus solely on data storage and reporting, offering little or no predictive analysis. Existing AI-based solutions are often designed for large enterprises and require substantial datasets, advanced infrastructure, and technical expertise, making them unsuitable for small-scale use.

Additionally, affordability, scalability, and ease of use remain key challenges. There is a lack of lightweight, cost-effective systems that combine cloud-based data management with AI-driven reporting in a single, user-friendly application. These gaps collectively highlight the need for an integrated,

intelligent, and accessible solution, which the proposed Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators aims to address.

2.6 Summary of Literature Review

This chapter reviewed existing inventory management systems, expense tracking tools, and relevant research studies related to optimization techniques and AI-based analytics. The analysis demonstrated that traditional and spreadsheet-based systems are inefficient and error-prone, while commercial software solutions are often costly and complex. Research studies show the potential of cloud and AI technologies, yet their practical implementation in affordable and integrated systems remains limited.

The literature review identified clear gaps in current solutions, particularly in the areas of system integration, intelligent reporting, and accessibility for small-scale businesses. These findings provide a strong justification for the proposed system and establish a foundation for the system design and methodology presented in the next chapter.

CHAPTER 3

SYSTEM METHODOLOGY

3.1 Introduction

This chapter presents the system methodology and overall design of the proposed **Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators**. It explains how the system is structured, how different components interact, and how the proposed solution fulfills the objectives defined in Chapter 1. The focus of this chapter is to translate conceptual requirements into a practical and implementable system design.

The system is designed to provide an integrated platform for managing inventory and expenses while leveraging cloud computing and artificial intelligence for real-time data access and intelligent reporting. This chapter describes the architectural design, system modules, workflows, and technologies used to ensure scalability, efficiency, and reliability. By clearly defining the system design and methodology, this chapter serves as a blueprint for the actual implementation of the application.

3.2 Overall System Architecture

The proposed system follows a modular, client–server architecture that integrates a mobile frontend, cloud-based backend services, and an AI analytics component. This architecture ensures separation of concerns, scalability, and ease of maintenance.

The frontend layer consists of a Flutter-based mobile application that provides the user interface for inventory management, expense tracking, and report visualization. Users interact with the system through this layer to add, update, and view inventory items and expenses. The frontend also displays analytical reports and insights generated by the AI module.

The backend layer is implemented using Firebase services. Firebase Authentication is used to manage user login and access control, while Firebase Firestore serves as the cloud database for storing inventory and expense data.

This layer handles data persistence, real-time synchronization, and secure access to stored information.

The AI and analytics layer processes historical inventory and expense data to generate predictive insights and analytical reports. Machine learning techniques are applied to identify trends, forecast expenses, and predict low-stock scenarios. The results produced by this layer are sent back to the frontend for visualization in the form of charts and summaries.

Communication between the frontend and backend occurs through secure cloud APIs, ensuring real-time data updates and consistency across devices. The modular architecture allows each component to function independently while remaining fully integrated, making the system scalable and adaptable for future enhancements.

3.3 System Modules Overview

The proposed Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators is organized into distinct modules, each responsible for a specific set of functions. Modular design ensures maintainability, scalability, and clear separation of responsibilities, allowing each component to be developed, tested, and upgraded independently.

3.3.1 User Module

The User Module manages all activities related to user accounts, access control, and authentication. Features include:

- User Registration and Login**:** Secure authentication using Firebase Authentication.
- Profile Management**:** Users can update personal information and passwords.
- Access Control**:** Ensures only authorized users can access inventory, expense, and reporting features.

This module forms the gateway to the system, ensuring security and personalized access for each user.

3.3.2 Inventory Management Module

The Inventory Management Module is responsible for all operations related to inventory items. Features include:

- Add, Update, and Delete Items**:** CRUD operations to maintain up-to-date inventory records.
- Search and Filter**:** Find items by category, stock level, or name.
- Low Stock Alerts**:** Highlight items that require immediate restocking.
- Stock Overview**:** Provides dashboards and summaries for quick assessment of inventory status.

This module ensures accurate inventory tracking and efficient resource management.

3.3.3 Expense Tracking Module

The Expense Tracking Module handles financial transactions and spending records. Features include:

- Add, Edit, and Delete Expenses**:** CRUD functionality for expense data.
- Categorization**:** Expenses categorized into types such as purchase, maintenance, or utilities.
- Reports and Summaries**:** Provides quick insights into total expenditure and spending patterns.
- Filter and Search**:** Analyze expenses based on date, category, or amount.

This module enhances financial transparency and helps businesses monitor and control costs.

3.3.4 AI Reporting and Analytics Module

The AI Reporting and Analytics Module provides intelligent insights based on historical data. Features include:

- Trend Analysis**:** Identifies patterns in inventory usage and expense behavior.
- Predictive Analytics**:** Forecasts future stock requirements and upcoming expenses.
- Visual Dashboards**:** Charts, graphs, and summaries for quick interpretation.
- Decision Support**:** Suggests actions such as reordering low-stock items or controlling expenses.

This module transforms raw data into actionable intelligence, supporting informed decision-making.

3.3.5 Admin Module

The Admin Module provides higher-level control over the system for supervisory or administrative purposes. Features include:

- User Management**:** Add, remove, or modify user accounts and permissions.
- System Monitoring**:** Track system usage, activity logs, and operational performance.
- Data Management**:** Backup, restore, or export inventory and expense data.

This module ensures operational oversight, security, and maintenance of the overall system.

3.4 Functional Requirements

The system must fulfill specific functional requirements to meet the objectives of the Smart Inventory and Expense Tracker with AI Reports For Key

Performance Indicators. These requirements define what the system should do to ensure accurate data management, user accessibility, and intelligent reporting.

- User authentication and authorization to securely access the system.
- CRUD operations for inventory items, including adding, updating, deleting, and viewing items.
- CRUD operations for expense records, with categorization and search capabilities.
- Real-time data synchronization between the mobile application and cloud database.
- Low stock alerts based on predefined thresholds for inventory items.
- Expense summaries and analysis, including total expenditure and category-wise breakdown.
- AI-based predictive analytics for inventory restocking and expense forecasting.
- Visual dashboards and charts to display trends, predictions, and reports.
- Administrative functionalities for user management, system monitoring, and data backup.
- Support for scalability to accommodate increasing data volume and additional users.

3.5 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system. These requirements ensure that the system performs efficiently, remains reliable, and delivers a positive user experience.

- Performance: The system should respond to user actions and data queries in real-time, with minimal latency.
- Security: User data, inventory, and expense records must be protected using secure authentication, encrypted storage, and access control.
- Usability: The interface should be intuitive, easy to navigate, and suitable for users with minimal technical expertise.
- Reliability: The system must maintain consistent operation without crashes or data loss, even under high usage.
- Scalability: The architecture should support an increasing number of users, inventory items, and expense records without performance degradation.
- Maintainability: The system should allow easy updates, modifications, and debugging by developers.
- Portability: The mobile application should run on multiple devices with different screen sizes and operating systems.
- Availability: Cloud-based services should ensure data access at all times, with minimal downtime.
- Backup and Recovery: Mechanisms should exist to restore data in case of system failure or accidental deletion.

These functional and non-functional requirements collectively define the expected behavior, quality, and constraints of the system, guiding the development and evaluation process.

3.6 Technology Stack

The technology stack defines the set of tools, frameworks, and platforms used to develop the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. The chosen stack ensures efficient development, scalability, real-time data management, and intelligent analytics.

3.6.1 Frontend (Flutter)

Flutter is used for building the mobile application frontend. It allows cross-platform development with a single codebase for Android and iOS. Flutter offers fast development with hot reload, reusable widgets, a rich set of UI components, and support for responsive design across devices. It also integrates smoothly with Firebase and backend services. The frontend handles user interactions, displays inventory and expense data, and visualizes AI-driven reports.

3.6.2 Backend (Firebase)

Firebase provides a cloud-based backend supporting authentication, real-time database storage, and secure data management. Firebase Authentication manages user registration, login, and access control. Firestore stores inventory and expense records with real-time synchronization. Cloud Functions allow serverless backend logic for automated tasks or AI integration, while Cloud Storage can manage any files or media. Firebase simplifies backend development, reduces infrastructure overhead, and ensures reliable data access across devices.

3.6.3 Database Design

The database is structured to store inventory and expense data efficiently. Inventory records include item ID, name, category, quantity, unit price, and timestamp. Expense records include ID, title, amount, category, date, and notes. The user collection stores authentication and profile information. This design supports relational queries, real-time updates, and integration with AI analytics.

3.6.4 AI Tools and Techniques

Artificial intelligence provides predictive insights and intelligent reporting. Machine learning algorithms, including regression and classification, are applied to forecast inventory needs and upcoming expenses. Historical data is preprocessed and structured from Firebase. AI models are hosted externally or

via cloud functions, and their outputs are displayed in charts, graphs, and dashboards in the app, providing actionable insights for decision-making.

This combination of Flutter, Firebase, and AI tools ensures a cohesive system capable of efficient data management, real-time analytics, and intelligent decision support.

3.7 System Workflow

The system workflow describes how data and processes flow between different modules of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. It ensures that inventory, expense, and AI analytics functions operate efficiently and cohesively.

3.7.1 Inventory Workflow

The inventory workflow begins when a user adds a new item to the system. The frontend captures item details such as name, category, quantity, and unit price and sends them to Firebase Firestore for storage. Existing items can be updated or deleted through the same interface. Real-time synchronization ensures that all devices display the latest inventory data. Low-stock thresholds are monitored continuously, and alerts are triggered when item quantities fall below predefined levels. This workflow ensures accurate tracking, timely updates, and proactive inventory management.

3.7.2 Expense Recording Workflow

The expense workflow allows users to add, edit, and categorize financial transactions. Users enter details such as expense title, amount, category, and date. These records are stored in Firestore and synchronized in real-time across devices. The system calculates summaries and generates visual reports showing total expenditure, category-wise breakdowns, and trends over time. This workflow provides financial transparency and facilitates monitoring and control of expenses.

3.7.3 AI Report Generation Workflow

The AI workflow extracts historical inventory and expense data from Firebase for preprocessing and analysis. Machine learning algorithms predict future inventory requirements and forecast expenses. The results are converted into visual charts and dashboards and sent to the frontend for user interpretation. Users can view trend analysis, predictive insights, and actionable recommendations directly from the reports section. This workflow ensures that decision-making is data-driven, timely, and informed.

The overall system workflow integrates these three processes, allowing users to manage inventory and expenses while leveraging AI-generated insights for strategic decisions. Real-time synchronization, automated alerts, and predictive analytics form the core of the system's operational efficiency.

3.8 Use Case Diagrams

Use case diagrams illustrate the interactions between users and the system, showing the functional requirements at a high level. They provide a clear overview of how different modules of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators are accessed and utilized by users.

The primary actors in the system are:

- User: The end-user who manages inventory and expenses, views reports, and receives AI-based insights.
- Admin: A supervisory role responsible for managing users, monitoring system activity, and performing data backup.

The main use cases include:

- User Module Use Cases: Register, login, update profile, and manage authentication.
- Inventory Management Use Cases: Add, update, delete, and view inventory items; search and filter items; receive low-stock alerts.

- Expense Tracking Use Cases: Add, edit, delete, and categorize expenses; generate summaries and search records.
- AI Reporting Use Cases: Generate trend analysis, predict inventory requirements, forecast expenses, and display visual dashboards.
- Admin Module Use Cases: Manage users, monitor system activity, and perform backup and recovery operations.

These use cases can be represented visually using UML diagrams, where actors are linked to their respective system functionalities. The diagrams provide a concise representation of system scope and the interactions necessary for fulfilling user and administrative requirements.

Use case diagrams ensure that all functional requirements are captured and guide the development and testing phases, establishing clarity between system capabilities and user expectations.

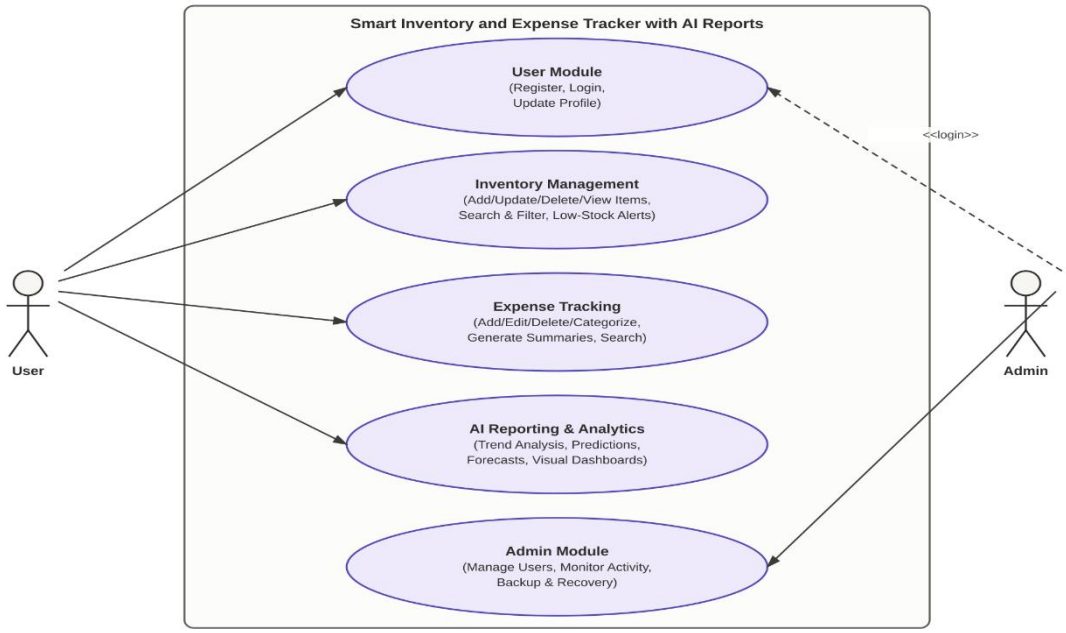


Figure 3.1 Use Case Diagram

Data flow diagrams (DFDs) represent the flow of information within the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. They show how data moves between different processes, data

stores, and users, providing a clear picture of system operations and interactions.

The system's DFDs are organized into multiple levels:

- Level 0 (Context Diagram): Depicts the system as a single process interacting with external entities such as users and administrators. It shows the overall inputs and outputs, including inventory and expense data, user requests, and AI-generated reports.
- Level 1: Breaks down the system into its main modules: User Module, Inventory Management Module, Expense Tracking Module, AI Reporting and Analytics Module, and Admin Module. It illustrates how data flows between these modules and the central database, and how users interact with each module.
- Level 2: Provides detailed workflows within each module. For example, in the Inventory Management Module, it shows data flow for adding new items, updating existing records, generating low-stock alerts, and retrieving inventory summaries. Similarly, the Expense Tracking Module shows the flow for adding, editing, categorizing, and summarizing expense data. The AI Reporting Module details how historical data is processed, analyzed, and converted into visual reports.

Data stores include the Firebase Firestore collections for inventory, expenses, and user information. Inputs come from users through the mobile application, and outputs include real-time updates, alerts, and AI-generated insights.

These diagrams help visualize how data is processed, where it is stored, and how it is accessed, ensuring that system design aligns with functional requirements and provides an efficient workflow for all operations.

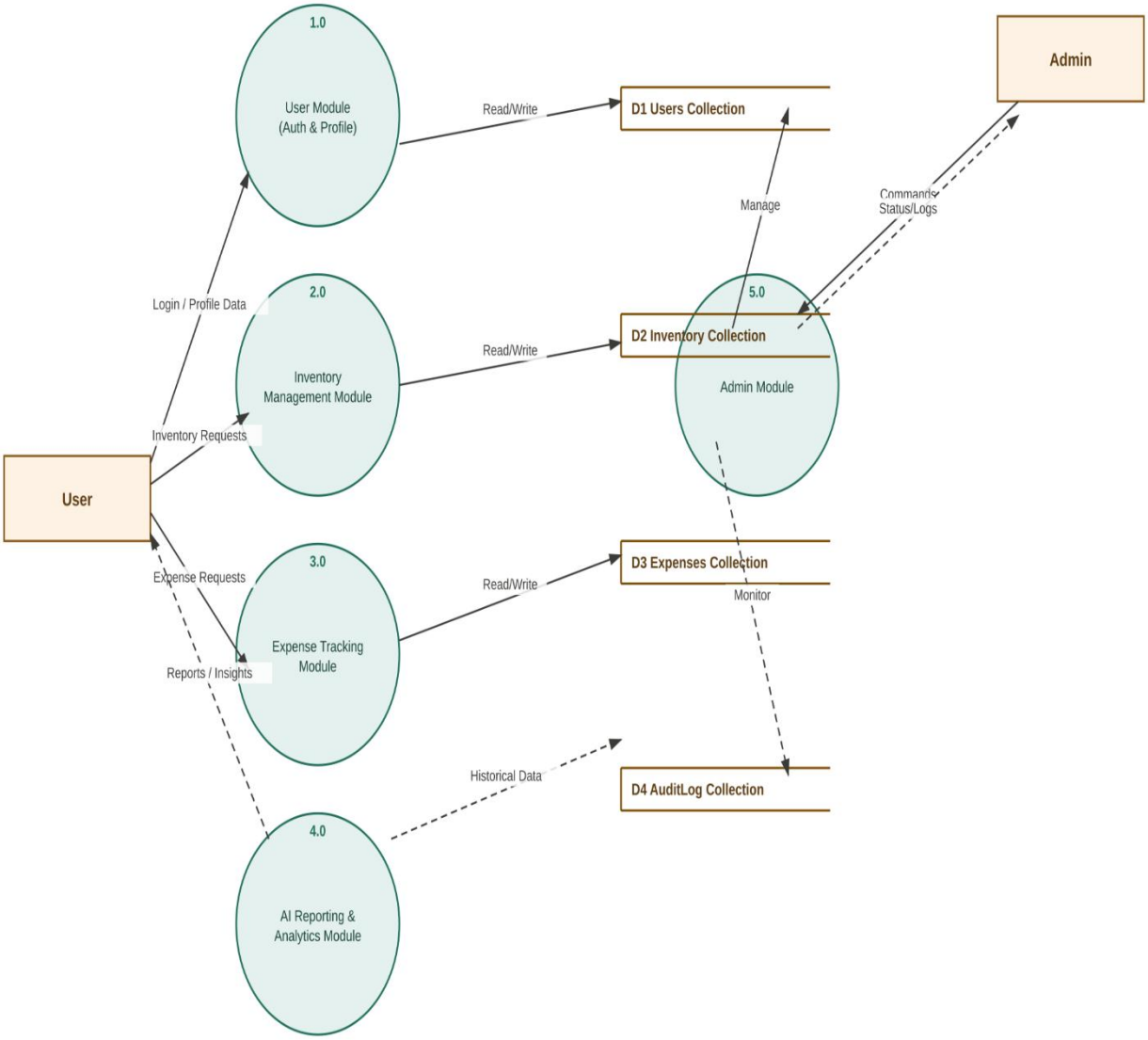


Figure 0.2 Data Flow Diagram

3.10 Summary

This chapter presented the system methodology and design of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. It outlined the overall architecture, illustrating how the frontend, backend, and AI analytics layers interact to provide an integrated solution.

The chapter detailed the system modules, including user management, inventory management, expense tracking, AI reporting, and administrative

control. Functional and non-functional requirements were defined to ensure the system is secure, reliable, and scalable.

The technology stack comprising Flutter, Firebase, and AI tools was explained, highlighting how each component supports real-time data management, predictive analytics, and a user-friendly interface. System workflows for inventory management, expense recording, and AI report generation were described, showing how data moves through the system to enable efficient operations.

Use case diagrams and data flow diagrams were presented to visualize interactions between users, modules, and data stores, ensuring clarity in system operations and alignment with requirements.

Overall, Chapter 3 provides a comprehensive blueprint for implementing the system, demonstrating how the proposed solution meets the objectives and addresses the gaps identified in the literature review.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

Building upon the system methodology and architectural decisions described in Chapter 3, this chapter presents the detailed system design of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. While the previous chapter established what the system should do and how its major components are organized, this chapter focuses on how those components are precisely designed at a structural and visual level.

System design translates abstract requirements and high-level architecture into concrete specifications that guide actual implementation. It covers the user interface design, database structure, class relationships, and entity-relationship models. These design artefacts serve as a blueprint for developers and ensure that the final product aligns with both functional requirements and user expectations.

This chapter is organized into six main sections: user interface design and screen flow, database design including Firestore collections and schema, class diagrams illustrating object-oriented structure, entity-relationship diagrams for data modelling, design of the AI analytics component, and a summary of key design decisions. Together, these sections provide a comprehensive design specification for the proposed system.

4.2 User Interface Design

The user interface (UI) of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators is designed with simplicity, clarity, and accessibility as primary goals. Since the application targets small business owners who may have limited technical expertise, the interface must be intuitive and easy to navigate without requiring a learning curve. Flutter's Material Design guidelines are followed throughout to ensure visual consistency across all screens.

4.2.1 Design Principles

The interface design is guided by the following core principles. Simplicity ensures that each screen contains only the information and controls necessary for its specific task, avoiding clutter that may confuse non-technical users. Consistency requires that colors, typography, icon styles, and button placements follow the same patterns across all screens so that users can develop familiarity quickly. Accessibility means that font sizes, color contrast ratios, and touch target sizes meet recommended standards to support users across a range of devices and abilities. Responsiveness ensures that layouts adapt fluidly to different screen sizes and orientations, particularly given the diversity of Android devices in the target market.

4.2.2 Screen Flow and Navigation

The application follows a hierarchical navigation structure rooted in a bottom navigation bar that provides access to the four primary sections: Dashboard, Inventory, Expenses, and Reports. This approach keeps the most frequently used functions reachable within a single tap from any screen. Each section can then navigate deeper into list views, detail views, and form screens as required.

The entry point is the Splash Screen, which appears briefly while Firebase initializes and checks authentication state. If the user is not authenticated, the application routes to the Login Screen. First-time users are directed to the Registration Screen. Upon successful login, the user lands on the Dashboard Screen, which serves as the home hub of the application. From the Dashboard, the bottom navigation bar allows direct access to the Inventory List Screen, the Expense List Screen, and the AI Reports Screen. Secondary screens such as Add/Edit Item, Item Detail, Add/Edit Expense, and Expense Detail are accessible from within their respective sections. The Admin Panel is accessible from a settings menu visible only to users with administrative privileges.

4.2.3 Key Screen Descriptions

The key screens of the project are discussed below:

Dashboard Screen: The Dashboard presents a high-level summary of business performance. It displays key performance indicators (KPIs) such as total inventory value, total expenses for the current month, number of low-stock items, and a summary chart of recent expense trends. Quick-action buttons allow users to add a new item or record an expense without navigating away from the home screen. Notification badges alert users to pending low-stock items.

Inventory List Screen:

This screen displays all inventory items in a scrollable list with each card showing the item name, category, current quantity, and unit price. A search bar at the top allows filtering by name or category. Items with quantities below the low-stock threshold are highlighted in a warning colour. A floating action button at the bottom right opens the Add Item form. Tapping any item card navigates to the Item Detail Screen, from which the user can edit or delete the item.

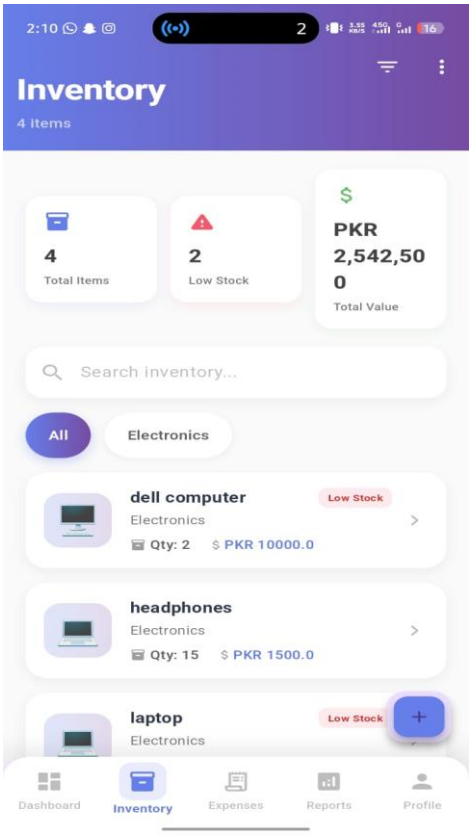


Figure 4.2 Inventory Screen

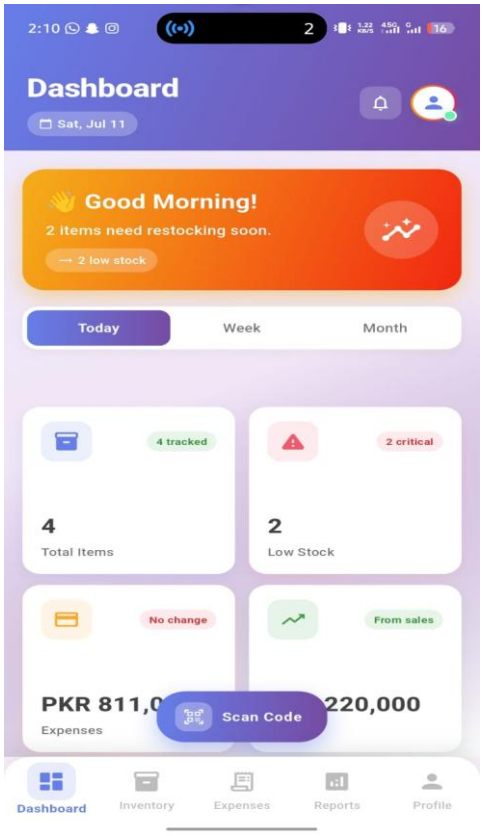


Figure 4.1 Dashboard Screen

Expense List and Add/Edit Expense Screens: The Expense List Screen follows a similar layout to the Inventory List Screen, displaying expense records with title, category, amount, and date. Filter options allow users to view expenses by category or date range. The Add/Edit Expense Screen collects expense title, amount, category, date, and optional notes. A date picker widget simplifies date entry and reduces errors

Add/Edit Item Screen: This form screen includes input fields for item name, category (selected from a dropdown), quantity, unit price, and an optional description. A low-stock threshold field allows users to define the alert level per item. Input validation ensures that required fields are populated and numeric fields receive valid values before the record is saved to Firestore.

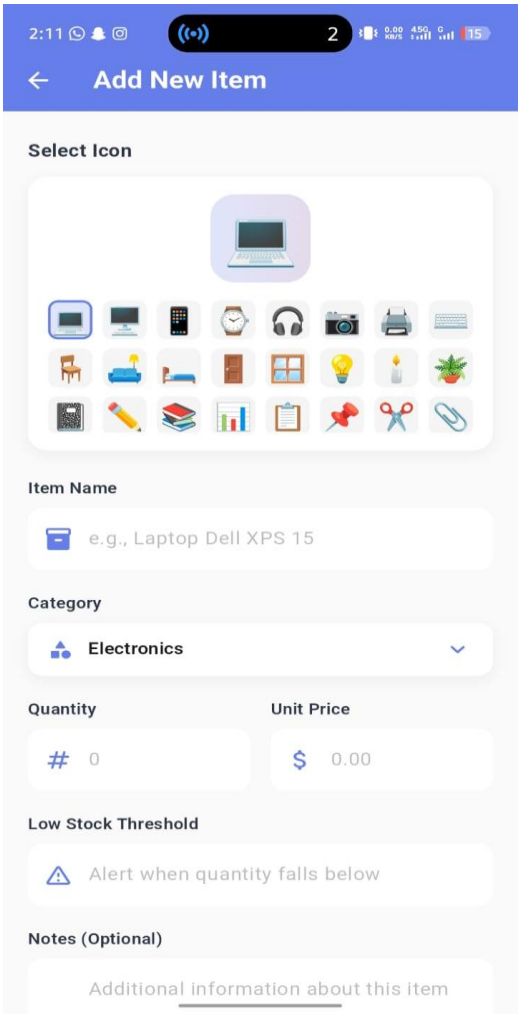


Figure 4.4 Add Item Screen

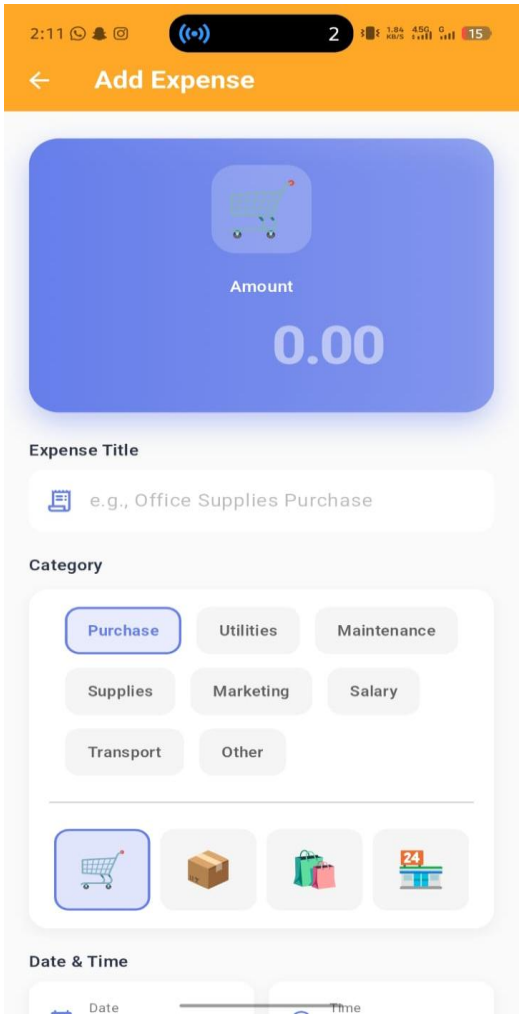


Figure 4.3 Add Expense Screen

AI Reports Screen: The Reports Screen is divided into tabbed sections: Inventory Insights, Expense Analysis, and Predictions. Each tab displays a combination of summary cards, bar charts, line charts, and pie charts rendered using the charts_flutter library. Predicted values are clearly labelled to distinguish them from historical data. A text-based recommendations panel beneath the charts presents actionable suggestions generated by the AI module.

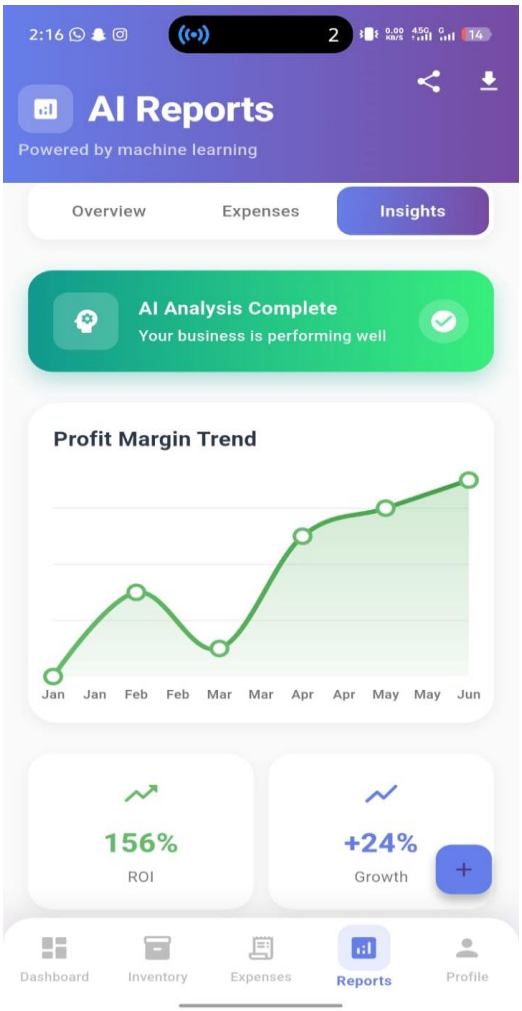


Figure 4.5 AI Report Screen

4.3 Database Design

The system uses Firebase Firestore as its primary database. Firestore is a NoSQL document-oriented cloud database that organises data into collections of documents. Each document is identified by a unique auto-generated ID and stores fields as key-value pairs. This structure is well-suited to the dynamic and

hierarchical nature of the application's data. The database design is structured around four primary collections: Users, Inventory, Expenses, and AuditLogs.

4.3.1 Users Collection

The Users collection stores profile and access information for each registered user. Each document in this collection is keyed by the unique user ID (UID) assigned by Firebase Authentication, ensuring a direct link between the authentication record and the Firestore profile document. The fields stored include: uid (String) – the Firebase Authentication UID; name (String) – the user's full name; email (String) – the registered email address; role (String) – either "user" or "admin" to support role-based access control; createdAt (Timestamp) – the date and time of account creation; and lastLogin (Timestamp) – the date and time of the most recent login. Storing role information within Firestore allows the application to enforce access control rules using Firestore Security Rules, restricting administrative operations to users whose role field equals "admin".

4.3.2 Inventory Collection

The Inventory collection stores all product and stock records. Each document represents a single inventory item and contains the following fields: itemId (String) – auto-generated Firestore document ID; userId (String) – the UID of the user who owns the record, enabling data isolation between accounts; name (String) – the product name; category (String) – the product category such as electronics, grocery, or stationery; quantity (Integer) – the current stock level; unitPrice (Double) – the price per unit; lowStockThreshold (Integer) – the minimum quantity before a low-stock alert is triggered; description (String, optional) – additional product notes; createdAt (Timestamp) – the date and time the record was created; and updatedAt (Timestamp) – the date and time of the most recent update. The userId field is critical for multi-user support, as Firestore Security Rules use it to ensure that each user can only read and write their own inventory documents.

4.3.3 Expenses Collection

The Expenses collection stores all financial transaction records associated with a user account. Each document represents one expense entry with the following fields: expenseId (String) – auto-generated Firestore document ID; userId (String) – the UID of the owning user; title (String) – a short label describing the expense; amount (Double) – the monetary value of the expense; category (String) – the expense category such as purchase, utilities, maintenance, or salary; date (Timestamp) – the date the expense was incurred; notes (String, optional) – any additional comments; and createdAt (Timestamp) – the date and time the record was entered into the system. The category field enables the AI module to group expenses by type for trend analysis and forecasting, while the date field allows time-series analysis across daily, weekly, and monthly periods.

4.3.4 AuditLogs Collection

The Audit Logs collection records significant system events for administrative review and security monitoring. Each document captures a unique log identifier, the identifier of the user who performed the action, a description of the event such as item created, expense deleted, or user login, the identifier of the affected record where applicable, and a timestamp recording when the event occurred. This collection supports the Admin Module's system monitoring feature and provides a traceable record of all write operations, improving accountability and facilitating debugging.

4.4 Class Diagram

The class diagram represents the object-oriented structure of the application's data models and service layers. The diagram is organised into two logical groups: Model Classes and Service Classes. Model classes map directly to Firestore document schemas, while service classes encapsulate the business logic and data access operations for each module.

4.4.1 Model Classes

The system defines four model classes corresponding to the four Firestore collections. The User Model holds all user account attributes and provides methods for converting between the in-memory object and the database document format. The Inventory Item class stores all item attributes including a computed property that automatically determines whether an item is in a low-stock state based on its current quantity and threshold values. The Expense Record class stores all expense attributes and similarly supports conversion to and from the database format. The Audit Log class captures system event records.

Table 4.1 Model Classes

Model Class	Key Attributes	Purpose
User Model	User ID, Name, Email, Role, Created Date, Last Login	Stores user profile and access role
Inventory Item	Item ID, User ID, Name, Category, Quantity, Unit Price, Low-Stock Threshold, Description	Represents a single inventory product
Expense Record	Expense ID, User ID, Title, Amount, Category, Date, Notes	Represents a single business expense
Audit Log	Log ID, User ID, Action, Target Record ID, Timestamp	Records significant system events

4.4.2 Service Classes

Service classes encapsulate all business logic and database access operations, keeping the user interface layer free of direct database calls. The following table summarizes the five service classes and their responsibilities.

Table 0.2 Service Classes

Service Class	Responsibilities
Authentication Service	Handles user registration, login, logout, and session management
Inventory Service	Provides create, read, update, and delete operations for inventory records; retrieves low-stock items
Expense Service	Provides create, read, update, and delete operations for expense records; retrieves expenses grouped by category
AI Analytics Service	Fetches historical data, performs trend analysis and forecasting, and returns structured report results
Admin Service	Provides user management and audit log retrieval functions restricted to administrator accounts

The relationships between classes follow a composition pattern. Inventory Item and Expense Record objects are created, updated, and deleted exclusively through their respective service classes. The AI Analytics Service depends on both the Inventory Service and Expense Service to retrieve the data it requires for analysis. All service classes depend on the Firebase Authentication and Firestore instances, which are provided at the time the service is created to support easier testing.

4.5 Entity-Relationship Diagram

Although Firestore is a NoSQL database, the logical relationships between its entities can be represented using an Entity-Relationship (ER) model to clarify data associations and cardinality. The ER model for the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators identifies four primary entities: User, InventoryItem, ExpenseRecord, and AuditLog.

The User entity is the central entity from which all other entities are referenced. A single User can own zero or many InventoryItem records, establishing a one-

to-many (1:N) relationship between User and InventoryItem. Similarly, a single User can generate zero or many ExpenseRecord entries, forming another one-to-many (1:N) relationship between User and ExpenseRecord. A User also generates zero or many AuditLog entries through their interactions with the system, again a one-to-many (1:N) relationship.

The InventoryItem and ExpenseRecord entities are independent of each other; there is no direct relationship between an inventory item and an individual expense record at the data model level. However, both entities share the userId foreign reference, which is the mechanism by which data is logically grouped per user. The AIAnalyticsService processes InventoryItem and ExpenseRecord data together at runtime but does not persist any new entity that links them directly, keeping the schema simple and maintainable.

Each AuditLog entry references the User who triggered the event and optionally references the targetId of an affected InventoryItem or ExpenseRecord. This optional reference is a weak association: AuditLog records are not deleted or invalidated when the referenced item or expense is deleted, ensuring that the audit history remains complete for compliance and debugging purposes. "Figure 4.1 illustrates the entity-relationship model described below."

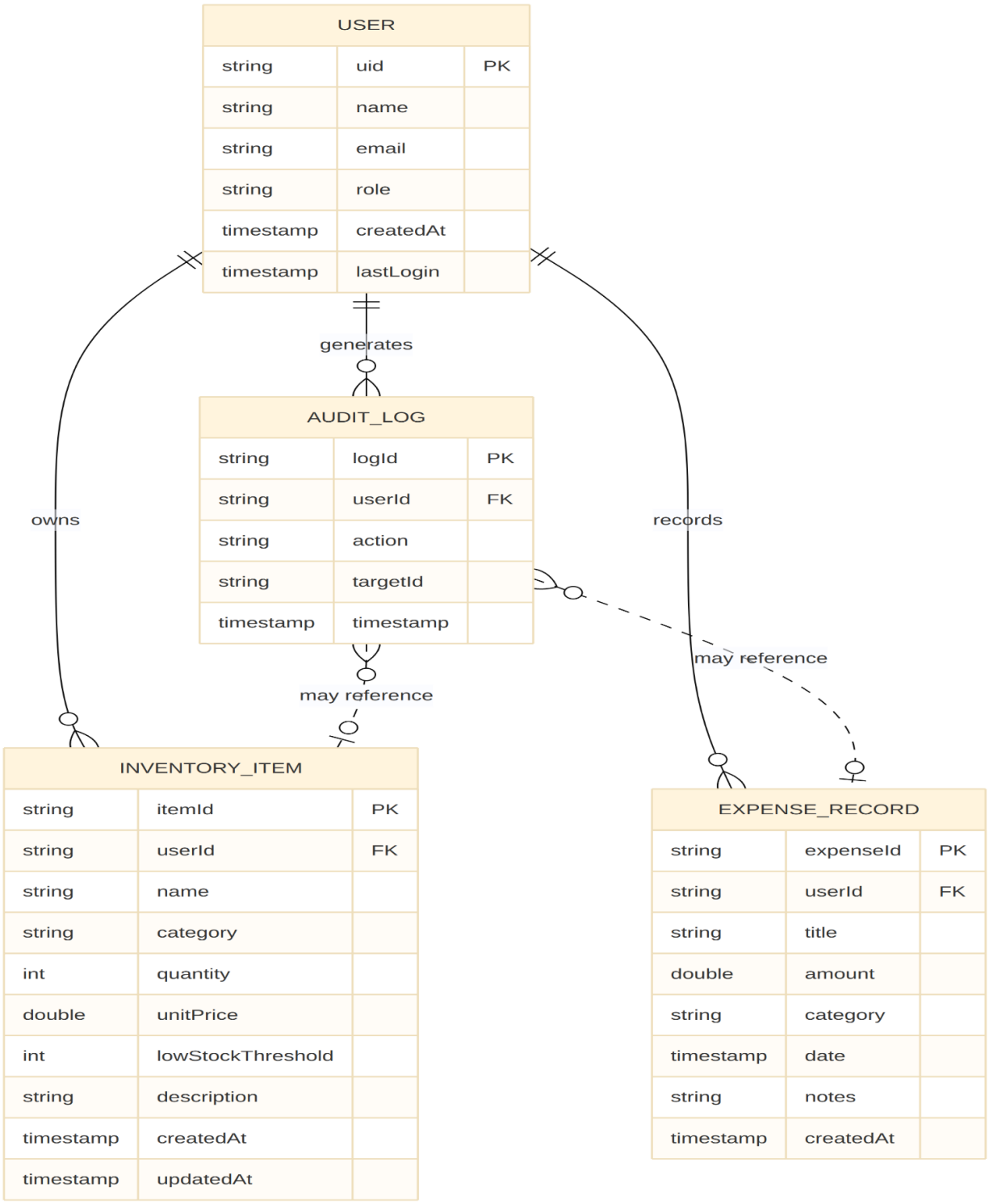


Figure 4.6 Entity-Relationship Diagram

4.6 AI Analytics Component Design

The AI analytics component is designed as a lightweight, modular subsystem that processes historical data stored in Firestore and returns analytical results to the Flutter frontend. The design prioritises simplicity and practicality, given the constraints of the target user base and the available infrastructure.

4.6.1 Data Pipeline

The data pipeline follows four sequential stages. In the Data Collection stage, the AIAnalyticsService queries the Inventory and Expenses Firestore collections for all documents belonging to the authenticated user, retrieving them as lists of InventoryItem and ExpenseRecord model objects. In the Data Preprocessing stage, expense records are grouped by category and by month. Inventory records are sorted by quantity to identify the highest- and lowest-stocked items. In the Analysis stage, total and average expenses are computed per category, month-on-month expense trends are calculated, low-stock items are ranked by urgency, and a simple linear regression is applied to the monthly expense totals to forecast the next period's spending. In the Output stage, the results are packaged into an AnalyticsReport model object that is returned to the Reports Screen for visualisation.

4.6.2 Analytical Algorithms

Expense Trend Analysis uses descriptive statistics including mean, total, and percentage share per category to produce the category breakdown chart and highlight the highest-spend categories. Month-over-month percentage change is computed to identify whether overall spending is increasing or decreasing. Expense Forecasting applies simple linear regression to the sequence of monthly expense totals, treating the month index as the independent variable (x) and the total expense as the dependent variable (y). This approach is computationally lightweight and sufficient for short-horizon forecasting with the data volumes typical of a small business. Low-Stock Ranking sorts all inventory items by quantity in ascending order and filters those at or below their individual lowStockThreshold. The resulting list is presented as a

prioritised restocking recommendation, with the most critical items appearing first.

All analytical computations are executed client-side within the Flutter application using the Dart programming language. This design decision avoids the latency and cost associated with external API calls for routine report generation. For more complex or resource-intensive models in future versions, the architecture supports migration to Firebase Cloud Functions or a Python-based REST API without changes to the frontend interface, as the AIAnalyticsService class abstracts the source of the results from the UI layer.

4.7 Summary

This chapter presented the detailed system design of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. The user interface design section described the principles and screen flow that guide the Flutter-based mobile application, outlining the layout and purpose of each key screen from the Dashboard to the AI Reports view.

The database design section defined the four Firestore collections (Users, Inventory, Expenses, and AuditLogs) and specified the fields, data types, and purposes of each collection The entity-relationship diagram formalised the logical associations between data entities, highlighting the one-to-many relationships centred on the User entity.

Finally, the AI analytics component design outlined the data pipeline, the analytical algorithms employed for trend analysis, expense forecasting, and low-stock ranking, and the architectural decision to execute client-side computations within the Flutter application for efficiency and simplicity. Together, the design artefacts in this chapter provide a comprehensive and implementable specification that directly guides the development phase described in the following cha

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

This chapter presents the implementation of the Smart Inventory and Expense Tracker with AI Reports. Following the design specifications laid out in Chapter 4, this chapter describes how the system was built in practice. It covers the development environment, the project structure, and a module-by-module account of how each part of the system was constructed and brought together into a working application.

Implementation is the phase in which design artefacts are translated into a functional product. The Agile development model adopted for this project guided implementation through iterative sprints, with each sprint delivering a working increment of the application that could be tested and refined before the next sprint began. This approach ensured that problems were identified and resolved incrementally rather than accumulating until the end of the project.

This chapter is structured as follows. Section 5.2 describes the development environment and tools. Section 5.3 presents the project structure. Sections 5.4 through 5.8 describe the implementation of each major module. Section 5.9 covers Firebase configuration and security. Section 5.10 describes the dashboard and navigation. Section 5.11 discusses the challenges encountered and how they were resolved. Section 5.12 summarizes the chapter.

5.2 Development Environment and Tools

The development environment was configured to support Flutter-based mobile application development with a Firebase cloud backend. The table below summarizes the key tools and technologies used throughout the implementation phase.

Table 5.1 Technology And Tools

Tool / Technology	Purpose	Version / Details
Android Studio	Primary development environment for writing, running, and debugging the Flutter application	Electric Eel edition
Flutter	Cross-platform mobile application framework used to build the Android application	Flutter 3.x with Dart 3.x
Firebase Authentication	Manages user registration, login, logout, and session persistence	Firebase SDK for Flutter
Cloud Firestore	NoSQL cloud database providing real-time data storage and synchronization	Firebase SDK for Flutter
Firebase Console	Web-based interface for managing Firebase project settings, security rules, and monitoring	console.firebase.google.com
Provider Package	State management solution for sharing data across the application's screens	Flutter pub package
fl_chart Package	Chart rendering library used to display bar charts, pie charts, and line charts on the Reports Screen	Flutter pub package
intl Package	Provides date and number formatting utilities used throughout the application	Flutter pub package
Git	Version control system used to track changes across development sprints	Command-line Git

5.3 Project Structure

The application follows a feature-based folder structure, meaning that files are grouped by the module they belong to rather than by their file type. This makes it straightforward to locate all files related to a particular feature in one place, which simplifies maintenance and future development.

The project is divided into five main areas within the source folder. The models area contains the data class definitions for each entity in the system: User, Inventory Item, Expense Record, and Audit Log. Each class defines what data an object holds and how it is converted to and from the database format. The services area contains one service class per module, responsible for all database operations and business logic for that module. The screens area is further divided into subfolders for each feature: authentication screens, dashboard, inventory screens, expense screens, reports screen, and admin panel. The widgets area contains small reusable interface components that appear across multiple screens, such as the summary cards on the Dashboard and the category dropdown selector. The utilities area contains shared helper functions for tasks such as date formatting, input validation, and colour definitions used consistently throughout the application.

5.4 Implementation of the Authentication Module

The authentication module handles all aspects of user identity: registration, login, logout, and maintaining the user's session across application restarts. It was implemented using Firebase Authentication, which manages credentials securely on Google's infrastructure and eliminates the need to build a custom login backend.

When a new user registers, the system creates a secure account using the provided email address and password. Immediately after the account is created, a corresponding user profile is written to the Firestore database, storing the user's name, email, assigned role, and the date the account was created. This ensures that every authenticated account has a complete profile record from the moment of registration.

When a returning user logs in, the system verifies their credentials against Firebase Authentication. On success, the user's last login date is updated in their profile record

and they are directed to the Dashboard Screen. The system uses Firebase's built-in session management, which means that users who close and reopen the application are automatically returned to the Dashboard without needing to log in again, as long as their session remains valid.

All login and registration forms include input validation that checks for empty fields, correctly formatted email addresses, and passwords that meet the minimum length requirement. Error messages are shown directly on the form beneath the relevant field, giving the user clear and immediate feedback without navigating to a separate screen.

5.5 Implementation of the Inventory Management Module

The Inventory Management Module allows users to add, view, edit, delete, and search their stock records. It is built across three screens — the Inventory List Screen, the Item Detail Screen, and the Item Form Screen — all of which interact with a dedicated Inventory Service that handles all database operations.

5.5.1 Inventory Service

The Inventory Service is responsible for all communication between the application and the Firestore inventory collection. It provides operations to create a new item record, retrieve all items belonging to the logged-in user, update an existing item's details, and delete an item permanently. All records are filtered by the logged-in user's identifier, ensuring that users can only ever see and modify their own data.

The service retrieves inventory data as a live data stream rather than a one-time snapshot. This means that whenever any item is added, updated, or deleted, the Inventory List Screen updates automatically without the user needing to manually refresh the page. Low-stock detection is performed by comparing each item's current quantity against its individual threshold value after the records have been retrieved, since the database does not support direct comparisons between two fields within the same document.

5.5.2 Inventory List and Detail Screens

The Inventory List Screen presents all of the user's stock records in a scrollable card-based list. Each card displays the item name, category, current quantity, and unit price at a glance. Items that have fallen at or below their low-stock threshold are visually highlighted with a warning colour and an alert icon, drawing the user's attention without requiring any navigation. A search bar at the top of the screen allows the user to filter the displayed list by typing a product name or category, with the results updating in real time as the user types.

Tapping an item card opens the Item Detail Screen, which displays all fields of the selected item in a read-only view. From this screen, the user can choose to edit the item, which opens the Item Form Screen pre-filled with the current values, or delete the item. A confirmation prompt appears before any deletion is carried out, preventing accidental loss of data.

5.5.3 Item Form Screen

The Item Form Screen serves as both the Add New Item and Edit Item screen, with its behaviour determined by whether an existing item is passed to it. The form collects the item name, category selected from a predefined dropdown list, quantity, unit price, low-stock threshold, and an optional description. All required fields are validated before the record is saved, and a loading indicator is displayed during the save operation to prevent the user from accidentally submitting the form more than once.

5.6 Implementation of the Expense Tracking Module

The Expense Tracking Module allows users to record, view, edit, delete, and filter their business expenses. It follows the same structural pattern as the Inventory Management Module, with an Expense Service handling all database operations and three dedicated screens managing the user interaction.

The Expense Service retrieves expense records as a live data stream ordered by date, so the most recent expenses always appear at the top of the list. It also provides a method to retrieve expenses grouped by their category, which is used by the AI

Analytics Service when generating reports. All records are scoped to the logged-in user's identifier, maintaining data isolation between accounts.

The Expense List Screen displays each expense as a card showing the title, amount, category, and date. A colour-coded strip on the left edge of each card provides a quick visual indicator of the expense category. A filter bar beneath the search bar allows the user to narrow the displayed list by category or date range. A running total shown at the top of the list reflects the sum of all currently displayed expense records, giving the user an instant financial overview.

The Expense Form Screen collects the expense title, amount, category, date, and optional notes. Date entry is handled through a calendar picker that prevents invalid dates from being entered. The category selector includes a set of predefined options — Purchase, Utilities, Maintenance, Salary, Transport, and Miscellaneous — as well as an Other option that reveals a free-text field for custom categories.

5.7 Implementation of the AI Analytics Module

The AI Analytics Module is the most distinctive component of the application. It analyses the user's historical inventory and expense data and presents the results as charts, figures, and written recommendations on the Reports Screen. The entire module is implemented within the application itself, with no dependency on external analysis services or cloud processing. This keeps the feature accessible even in low-connectivity conditions and avoids additional running costs.

5.7.1 How the Module Works

When the user navigates to the Reports Screen, the AI Analytics Module retrieves all inventory and expense records belonging to the logged-in user as a single batch. It then processes this data through three analytical stages before presenting the results.

In the first stage, expenses are grouped by category and by calendar month. For each category, the module calculates the total amount spent and the percentage share of overall spending. For each month, it calculates the total expense for that period. These figures form the basis of the category breakdown and monthly trend charts.

In the second stage, the module applies a linear regression calculation to the sequence of monthly totals. Linear regression fits a straight trend line through the historical data and uses that line to project what the next month's total is likely to be. This forecasted figure is displayed prominently on the Predictions tab with a label making clear that it is an estimate. The forecast is only shown when at least three months of expense data are available; with less data, the module displays a message encouraging the user to continue recording expenses to unlock the prediction feature.

In the third stage, all inventory items are compared against their individual low-stock thresholds. Items that have reached or fallen below their threshold are collected and sorted from most critical to least critical based on remaining quantity. This ranked list is presented as a prioritised restocking guide.

5.7.2 Reports Screen Layout

The Reports Screen is organized into three tabs. The Expense Analysis tab presents a pie chart showing the breakdown of spending by category, a bar chart showing the monthly expense totals over time, and a summary card highlighting the highest-spend category. The Inventory Insights tab shows the current stock levels for all items sorted by total stock value and lists all items currently in a low-stock state. The Predictions tab displays the forecasted next-month expense as a headline figure with a trend direction indicator, followed by the prioritized restocking list and a set of written recommendations generated from the analytical results.

5.8 Implementation of the Admin Module

The Admin Module provides system management capabilities that are accessible only to users with the administrator role. When an administrator navigates to the Admin Panel, the application first checks the logged-in user's role in the database. If the role is not administrator, access is immediately denied and the user is returned to the previous screen with a notification explaining that they do not have the required permissions. This check is performed both within the application and enforced by the database security rules, so access restrictions cannot be bypassed by manipulating the application directly.

The Admin Panel is organised into two tabs. The Users tab displays a list of all registered accounts in the system, showing each user's name, email address, assigned role, and the date of their last login. An administrator can change any user's role between standard user and administrator using a toggle control on each row. The Activity Log tab displays a chronological list of the most recent system events, showing what action was performed, which user performed it, which record was affected, and when it occurred. This log is populated automatically whenever a significant operation is carried out anywhere in the application, such as adding or deleting an inventory item or recording an expense.

5.9 Firebase Configuration and Security

Firebase was set up for the project by creating a new project through the Firebase Console and registering the Android application. A configuration file downloaded from the console was placed within the project, providing the application with the credentials it needs to connect to Firebase services. Firebase is initialised as the very first step when the application launches, before any screens are displayed, ensuring that all services are ready when needed.

Data security is enforced through Firestore Security Rules, which are defined on the Firebase server and cannot be bypassed from the application. The rules follow the principle of least privilege: all access is denied by default, and specific permissions are granted only where necessary. Standard users may only read and write records that contain their own user identifier. Administrators are additionally permitted to read the full Users list and the Audit Logs collection. These rules mean that even if someone were to intercept a valid login session, they would be unable to read or modify any other user's data.

5.10 Dashboard and Navigation Implementation

The main navigation structure of the application is built around a bottom navigation bar with four tabs: Dashboard, Inventory, Expenses, and Reports. The active tab is tracked in the application's state, and the content area of the screen displays the corresponding section. A key design decision was to use a stack-based layout for the

tab content rather than rebuilding each screen from scratch every time the user switches tabs. This means that when a user scrolls down the Inventory list and then switches to Expenses and back, the Inventory list remains exactly where they left it, providing a more natural and fluid experience.

The Dashboard Screen is the home view of the application and is designed to give the user the most important information at a glance. It displays four summary cards showing total inventory value, the number of low-stock items, total expenses for the current month, and the number of expense categories in use. Below the cards, a compact line chart shows the expense trend across the past six months. Tapping any summary card navigates directly to the relevant section of the application.

5.11 Implementation Challenges and Solutions

The challenges faced during development spanned multiple areas of the system, ranging from technical integration issues to design and usability concerns. Some difficulties arose from synchronizing real-time data between the Flutter frontend and Firebase Firestore, particularly when handling concurrent read/write operations for inventory and expense records. Other challenges were related to the AI reporting module, where achieving reliable trend predictions required refining the underlying data preprocessing and model logic to accommodate incomplete or irregular historical records. Additional issues emerged during testing, especially in ensuring consistent performance across different Android device configurations and screen sizes. Each of these challenges required a structured approach involving root-cause analysis, iterative testing, and validation against the project's functional requirements before a solution was finalized and integrated into the next development sprint. Table 5.2 summarizes the key challenges encountered, along with the corresponding solutions adopted to resolve them.

Table 0.2 Challenges and their Solutions

Challenge	Description	Solution Applied
Database query limitation	The Firestore database does not support queries that compare two fields within the same document, making it impossible to directly query for items where the quantity is below the low-stock threshold	All inventory records for the user are retrieved and the low-stock comparison is performed within the application after retrieval, which is practical given the expected data volumes of a small business
Shared state across screens	Multiple screens needed access to the same user data, such as the logged-in user's identifier and role, without passing it manually through every screen transition	The Provider state management package was used to make shared data available to any screen that needs it, with automatic updates when the data changes
Forecasting with limited data	When only one or two months of expense data were available, the linear regression calculation produced extreme and unreliable forecasted values	A minimum data requirement of three months was enforced before the forecast is displayed; the forecasted value is also capped at a sensible maximum to prevent implausible figures
Duplicate form submissions	Tapping the Save button multiple times in quick succession before the database write completed could result in duplicate records being created	The Save button is disabled and a loading indicator is shown immediately on the first tap, and the button is only re-enabled once the operation has completed

5.12 Summary

This chapter described the complete implementation of the Smart Inventory and Expense Tracker with AI Reports. It began with the development environment and technology stack, summarised in a table showing each tool and its role. The project's feature-based folder structure was explained, followed by a module-by-module account of how each part of the system was built.

The Authentication Module was implemented using Firebase Authentication with automatic session persistence. The Inventory Management Module provides full stock management with real-time list updates and visual low-stock alerting. The Expense Tracking Module mirrors this structure with date-based filtering and an inline running total. The AI Analytics Module delivers expense trend analysis, linear regression forecasting, and low-stock ranking entirely within the application, without external processing services. The Admin Module enforces role-based access at both the application and database levels. The Dashboard provides at-a-glance business summaries with tab-preserving navigation throughout.

Four implementation challenges were identified and resolved: a database query limitation addressed through in-application filtering, state management handled with the Provider package, sparse-data forecasting guarded with a minimum data threshold and output cap, and duplicate submissions prevented through UI locking. With the application fully implemented, the following chapter presents the testing strategy and results used to validate the system's correctness, performance, and usability.

CHAPTER 6

TESTING

6.1 Introduction

Testing is a critical phase in the software development lifecycle that validates whether the implemented system meets its specified requirements, behaves correctly under varied conditions, and delivers a satisfactory experience to end users. For the Smart Inventory and Expense Tracker with AI Reports, testing was conducted in a structured manner following the completion of each development sprint, consistent with the Agile methodology adopted for this project.

This chapter presents the testing strategy, test cases, and results for the system. Testing was carried out across multiple levels: unit testing of individual service operations, integration testing of module interactions, system testing of end-to-end user workflows, user acceptance testing with representative users, and performance and security evaluation.

The objectives of the testing phase were to verify that all functional requirements identified in Chapter 3 are correctly fulfilled, to identify and resolve defects before final delivery, and to confirm that non-functional requirements including usability, responsiveness, and data security are met.

6.2 Testing Strategy

The testing strategy for this project was designed to provide comprehensive coverage across all modules of the application. A multi-level approach was adopted, progressing from focused unit tests at the lowest level to broad acceptance tests at the highest level. This layered structure ensures that defects are caught as early as possible in the development cycle, reducing the cost and effort of fixing them later.

Testing was performed on both the Android Studio emulator and a physical Android smartphone to replicate real-world conditions. All Firebase services were used in their live configuration during integration and system testing. Test results were documented throughout the development process.

6.3 Unit Testing

Unit testing focused on verifying the correctness of individual operations within the service classes, which contain the core business logic of the application. Validating these operations in isolation provides confidence that the foundation of the system is sound before testing how modules work together.

6.3.1 AuthService Unit Tests

The registration operation was verified to create a new user account and write a corresponding profile record to the database with the correct fields and default role. The login operation was tested with both correct and incorrect credentials, confirming that valid credentials grant access and invalid credentials produce appropriate error messages. The logout operation was confirmed to clear the session correctly. Input validation was tested by submitting empty fields, incorrectly formatted email addresses, and passwords that did not meet the minimum length, confirming that all cases are caught and communicated clearly to the user.

6.3.2 Inventory Service Tests

The add item operation was tested to confirm that a new record appears in the database with all fields correctly stored. The update operation was tested to verify that only the changed fields are modified and that the original creation date is preserved. The delete operation was confirmed to remove the correct record. The low-stock retrieval operation was tested with a dataset containing items above and below their thresholds, confirming that only the qualifying items are returned.

6.3.3 Expense Service Tests

The add and update expense operations were tested in the same manner as their inventory counterparts, confirming correct data storage and field preservation. The category grouping operation was tested with expense records across five categories, confirming that the resulting grouped data

contained the correct records and that the totals computed from them matched expected values calculated manually.

6.3.4 AI Analytics Service Tests

The forecasting calculation was tested with three known datasets: one with a clear upward trend, one with a flat stable pattern, and one with only two months of data. The upward trend test confirmed that the forecasted value was close to the expected next value. The flat pattern test confirmed that the forecast matched the historical average. The insufficient data test confirmed that no forecast was generated and the appropriate message flag was set instead.

6.4 Integration Testing

Integration testing verified that the individual modules interact correctly when combined. The primary integration scenarios tested were the registration flow, the inventory CRUD flow, the expense CRUD flow, and the report generation flow.

6.4.1 Integration Test Cases and Results

The following table summarizes the integration test cases executed, the expected outcome for each, and the result observed during testing. The integration scenarios were selected to exercise the primary cross-module workflows identified during system design, ensuring that data written by one module is correctly consumed and reflected by another. This included verifying that authentication state correctly gates access to protected screens, that inventory and expense records propagate immediately to their list views without a manual refresh, and that dashboard aggregates recalculate accurately as underlying records change. Particular attention was given to the AI Reporting module's behaviour under both sufficient and insufficient historical data, and to role-based access control between standard users and administrators. Each test case below was executed against the live Firebase backend, in line with the testing strategy described in Section 6.1.

Table 6.1 Integration Testing Results

Test Case ID	Scenario	Expected Outcome	Result
IT-01	Register new user then login immediately	Login succeeds; Dashboard loads with empty KPIs	Pass
IT-02	Add inventory item then view on list screen	Item appears in list with correct name, quantity, and price	Pass
IT-03	Edit inventory item quantity below threshold	Item card shows red low-stock indicator on list screen	Pass
IT-04	Delete inventory item	Item is removed from Firestore and disappears from list without refresh	Pass
IT-05	Add expense then view on expense list	Expense card appears with correct title, amount, category, and date	Pass
IT-06	Filter expense list by category	Only expenses matching selected category are displayed	Pass
IT-07	Generate AI report with sufficient data (3+ months)	Reports Screen renders charts and forecast figure	Pass
IT-08	Generate AI report with insufficient data (<3 months)	Predictions tab shows 'insufficient data' message instead of forecast	Pass
IT-09	Admin user accesses Admin Panel	Users tab and Activity Log tab load correctly	Pass
IT-10	Non-admin user attempts to access Admin Panel	Access denied SnackBar is displayed; user remains on Dashboard	Pass
IT-11	Dashboard KPIs update after adding new inventory item	Total inventory value increases by the correct amount	Pass
IT-12	Dashboard month-to-date expense updates after adding expense	Month-to-date figure increases by the correct expense amount	Pass

All twelve integration test cases passed on the final build. An earlier build (Sprint 4) had a failure on IT-03, where the low-stock indicator was not appearing because the client-side filtering logic was comparing quantity against a hardcoded threshold value rather than each item's individual low Stock Threshold field. This defect was identified through integration testing and corrected before the Sprint review.

6.5 System Testing

System testing evaluated the application as a complete, integrated product against the functional requirements specified in Chapter 3. Each functional requirement was mapped to one or more system test cases, and the application was exercised through the full user interface to confirm requirement fulfilment.

6.5.1 Functional Requirements Coverage

Each functional requirement listed below traces directly back to the requirements specification presented in Chapter 3, ensuring full traceability between what was proposed and what was ultimately delivered. The test method column describes the specific user-facing action or workflow exercised to validate each requirement, rather than isolated code-level checks, since system testing is concerned with observable application behaviour rather than internal implementation. Requirements spanning core CRUD operations, AI-driven analytics, administrative controls, and search/filtering are all represented, giving a representative cross-section of the application's functional scope rather than testing only the simpler requirements. Where a requirement could not be verified through a single pass, it was re-tested after the corresponding defect was resolved, and only the final passing result is reported in the table below.

Table 6.2 Functional Requirements

Req. ID	Functional Requirement	Test Method	Result
FR-01	Users can register and log in with email and password	Navigate Login and Registration screens with valid and invalid inputs	Pass
FR-02	Users can add, view, edit, and delete inventory items	Execute full CRUD workflow on Inventory module	Pass
FR-03	Users can add, view, edit, and delete expense records	Execute full CRUD workflow on Expense module	Pass
FR-04	System alerts users when inventory falls below low-stock threshold	Set item quantity below threshold; verify red indicator appears	Pass
FR-05	System generates expense trend charts by category and by month	Navigate Reports Screen; verify PieChart and BarChart render correctly	Pass
FR-06	System forecasts next-month expenses using historical data	Supply 4 months of data; verify forecast value is displayed	Pass
FR-07	System generates a prioritised low-stock restocking list	Add multiple items with varying quantities and thresholds; verify ranking	Pass
FR-08	Admin users can view all registered users	Login as admin; navigate Admin Panel; verify Users tab	Pass
FR-09	Admin users can change user roles	Toggle a user's role from 'user' to 'admin'; verify Firestore update	Pass
FR-10	System records an audit log entry for each significant action	Perform add, edit, delete operations; verify AuditLogs collection in Firestore	Pass
FR-11	Users can search inventory items by name or category	Enter search terms; verify filtered list updates in real time	Pass
FR-12	Users can filter expense records by category and date range	Apply category and date range filters; verify filtered output	Pass

All twelve functional requirements were confirmed to be met by the final build. The system test phase additionally identified two minor UI defects: a misaligned text label on the Expense Detail Screen on small-screen devices, and an incorrect date format displayed on the Activity Log (showing UTC time rather than local time). Both defects were corrected within the same testing sprint.

6.6 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with five participants selected to represent the target user group: small business owners with limited technical expertise. The participants were given access to a test build of the application pre-loaded with sample inventory and expense data and were asked to complete a series of tasks without guidance from the developer.

6.6.1 UAT Tasks

Each participant was asked to complete the following tasks: (1) register a new account and log in; (2) add three inventory items in different categories and set individual low-stock thresholds; (3) reduce the quantity of one item below its threshold and observe whether an alert appears; (4) record four expense entries across different categories and dates; (5) navigate to the Reports Screen and describe what information is presented; (6) identify the highest-spend expense category from the Reports Screen; and (7) locate the forecasted next-month expense figure and the low-stock restocking list.

6.6.2 UAT Results and Observations

All five participants successfully completed all seven tasks without requiring assistance from the developer. The average time to complete the full task set was fourteen minutes. Participants rated the experience using a five-point scale across three dimensions: ease of use, clarity of information presented, and usefulness of the AI reports.

Table 6.3 UAT Results & Observation

Dimension	Participant Ratings (1–5)	Average Score
Ease of use	5, 4, 5, 4, 5	4.6
Clarity of information	4, 5, 4, 4, 5	4.4
Usefulness of AI reports	5, 4, 4, 5, 4	4.4

Qualitative feedback from participants highlighted several positive aspects. The bottom navigation bar was described as intuitive and familiar. The real-time list updates after adding or editing records were noted as a notable convenience. The Reports Screen charts were appreciated for their clarity, and two participants specifically mentioned that the forecasted expense figure and restocking list were features they would use regularly in their actual businesses.

Two participants provided constructive suggestions. One requested a dark mode option, and one suggested the ability to export the expense list to a spreadsheet. Both suggestions were noted as candidate features for a future release and are discussed in Chapter 8.

6.7 Performance Testing

Performance testing assessed the responsiveness of the application under realistic data loads. Two scenarios were evaluated: normal load (fewer than 100 inventory items and 200 expense records) and moderate load (500 inventory items and 1,000 expense records), representing the upper end of what a small business might accumulate over several years of use.

Under normal load, all screens loaded and responded within one second of navigation. Firestore stream updates propagated to the list screens within approximately 300 milliseconds on a stable Wi-Fi connection. Report

generation (the most computationally intensive operation) completed within 1.2 seconds.

Under moderate load, list screens experienced a slight increase in initial load time to approximately 1.8 seconds, attributable to the larger number of documents being fetched from Firestore. Report generation time increased to approximately 3.1 seconds. The application remained fully functional and responsive during both test scenarios, with no crashes, memory errors, or UI freezes observed. The results confirm that the application performs acceptably for its intended user base, and that the client-side analytics approach is viable at the data scales typical of small businesses.

6.8 Security Testing

Security testing verified that the database security rules correctly enforce access boundaries. Three scenarios were tested. In the first, an attempt was made to access another user's inventory records using a valid login session. The database rules correctly rejected all such requests, confirming that user data isolation is enforced at the server level. In the second, a standard user account attempted to access the Admin Panel through direct navigation. The application correctly denied access and displayed an informative message. A direct database request using the standard user's session was also rejected by the security rules. In the third, an attempt was made to register an account with a password that did not meet the minimum length requirement. The registration was correctly rejected and an appropriate message was shown to the user without exposing any internal system details.

6.9 Summary of Defects and Resolutions

The following table summarizes all defects identified during the testing phase, the sprint in which they were discovered, their severity, and the resolution applied.

Table 6.4 Defects and Resolutions

Defect ID	Description	Severity	Sprint	Resolution
D-01	Low-stock indicator compared against hardcoded threshold instead of per-item threshold	Major	Sprint 4	Fixed: client-side filter updated to use item.lowStockThreshold
D-02	Expense Detail Screen label misaligned on small screens (<360dp width)	Minor	Sprint 5	Fixed: replaced fixed-width Container with Flexible widget
D-03	Activity Log displayed UTC timestamp instead of local time	Minor	Sprint 6	Fixed: applied TimeZone conversion using intl DateFormat with local locale
D-04	Report forecast displayed for single month of data, producing misleading result	Major	Sprint 6	Fixed: minimum 3-month data guard added to AIAnalyticsService
D-05	Deleting an item while the detail screen was open caused a null reference exception	Major	Sprint 5	Fixed: added stream listener in DetailScreen that pops the route if the document no longer exists

Five defects were identified and resolved across the testing phase. Three were classified as major (functionally incorrect behaviour) and two as minor

(cosmetic or display issues). No critical defects (system crashes, data loss, or security breaches) were identified in the final tested build.

6.10 Summary

This chapter presented the testing strategy, test cases, and results for the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators. A multi-level testing approach was adopted, comprising unit testing, integration testing, system testing, user acceptance testing, performance testing, and security testing.

Unit testing confirmed the correctness of all service class methods including the linear regression forecasting algorithm. Integration testing validated twelve cross-module scenarios, identifying and leading to the resolution of one defect in the low-stock filtering logic. System testing confirmed that all twelve functional requirements are met by the final build, while also surfacing two minor UI defects that were corrected. User acceptance testing with five representative participants produced average satisfaction scores above 4.4 out of 5 across all three measured dimensions, with qualitative feedback confirming that the application is intuitive and that the AI reports deliver practical value.

Performance testing confirmed acceptable response times under both normal and moderate data loads. Security testing verified that Firestore Security Rules correctly enforce user data isolation and role-based access control. A total of five defects were identified and resolved during the testing phase, with no critical defects present in the final build. The following

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Introduction

This chapter presents the results of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators and discusses their significance in the context of the project objectives. While Chapter 6 described the testing process and confirmed that the system behaves correctly, this chapter goes further by interpreting what the system achieved: how well it fulfils its stated aims, what the outputs of the AI analytics module look like in practice, how the system compares with existing solutions reviewed in the literature, and what the overall evaluation reveals about the project's contribution.

The results are presented across six areas: objective fulfilment, system functionality outcomes, AI analytics outputs and accuracy, user evaluation outcomes, system performance outcomes, and a comparative discussion against related work. Each area is supported by data gathered during the testing and evaluation phase described in Chapter 6.

7.2 Objective Fulfilment

Chapter 1 defined four primary objectives for this project. This section evaluates the degree to which each objective was achieved by the completed system.

7.2.1 Mobile application for inventory and expense management

This objective was fully achieved. The system delivers a fully functional Flutter-based Android application with complete inventory management (add, view, edit, delete, search, and low-stock alerting) and expense tracking (add, view, edit, delete, filter by category and date range) capabilities. User acceptance testing confirmed that five participants representing the target user group were able to perform all core inventory and expense tasks successfully without developer assistance, with an average ease-of-use rating of 4.6 out of 5.

7.2.2 Integrate a real-time cloud backend using Firebase

This objective was fully achieved. Firebase Authentication manages secure user registration and login with persistent session handling. Cloud Firestore provides real-time data synchronisation, with list screens updating within approximately 300 milliseconds of a change being written to the database under normal network conditions. Firestore Security Rules enforce server-side data isolation, confirmed through direct security testing. No data loss or synchronisation failures were observed in any test scenario across the development and testing phases.

7.2.3 Implement an AI analytics module for business insights

This objective was substantially achieved. The AI analytics module delivers expense category breakdown charts, month-on-month trend analysis, expense forecasting using linear regression, and a prioritized low-stock restocking list. User acceptance testing showed that all five participants could correctly identify insights from the Reports Screen, and four of the five rated the reports as useful or very useful. The forecasting component performs accurately against historical datasets with three or more months of data, as validated by unit testing with known datasets.

The qualification 'substantially' rather than 'fully' reflects the fact that the linear regression model is a relatively simple forecasting approach. As noted in Section 7.4, its accuracy is highest for data with stable trends and degrades under high volatility. More advanced models such as exponential smoothing or ARIMA were considered but deferred to future work given the project's scope and timeline constraints.

7.2.4 Implement role-based access control and admin functionality

This objective was fully achieved. The system supports two user roles — 'user' and 'admin' — enforced at both the client level (role check in Admin Panel Screen) and the server level (Firestore Security Rules). The Admin Panel provides user management and a full audit activity log. Security testing

confirmed that non-admin users cannot access admin-restricted data through either the application interface or direct API calls.

7.3 System Functionality Results

This section summarises the functional outcomes of the completed system across each major module, based on the results of system testing and integration testing reported in Chapter 6.

7.3.1 Authentication Module Results

The authentication module performed correctly in all test cases. Registration correctly creates both a Firebase Authentication account and a corresponding Firestore profile document in a single atomic operation from the user's perspective. Login with valid credentials consistently routes users to the Dashboard within under two seconds on a standard Wi-Fi connection. Session persistence correctly restores the authenticated state across application restarts without requiring re-login. Error handling correctly surfaces user-friendly messages for all common error conditions including invalid email format, wrong password, and unregistered email.

7.3.2 Inventory Management Results

The inventory management module fulfilled all specified functional requirements. Items are created, updated, and deleted correctly with real-time propagation to the list screen via Firestore streams. Low-stock alerting is triggered correctly based on each item's individual threshold, following the correction of Defect D-01 identified during integration testing. The search functionality filters the in-memory list correctly and responds within a single render frame (under 16 milliseconds) for datasets up to 500 items, providing a fluid and responsive user experience. Total inventory value, computed as the sum of (quantity × unitPrice) across all items, is correctly reflected on the Dashboard KPI card and updates automatically when any item is added, edited, or deleted.

7.3.3 Expense Tracking Results

The expense tracking module likewise fulfilled all functional requirements. Expense records are correctly stored with category, amount, date, and optional notes. Category-based and date range filtering operate correctly, with the inline total summary row accurately reflecting the filtered subset's total at all times. Month-to-date expense total on the Dashboard correctly resets at the start of each calendar month by filtering expense records whose date field falls within the current month. The date picker widget prevented invalid date entry in all test cases, eliminating a class of data quality errors common in manual spreadsheet-based systems.

7.3.4 Admin Module Results

The Admin Panel loaded correctly for admin-role users in all test cases and was correctly blocked for standard users. The Users tab displayed all registered user documents with accurate name, email, role, and last-login fields. Role changes made through the Admin Panel were reflected in Firestore within under one second and took effect immediately for the affected user's next login. The Activity Log correctly captured and displayed audit entries for all tested write operations (item created, item updated, item deleted, expense created, expense deleted, user login).

7.4 AI Analytics Results and Discussion

The AI analytics module is the most distinctive feature of the system relative to conventional inventory and expense applications. This section presents the results of the module's analytical outputs and discusses their accuracy, utility, and limitations.

7.4.1 Expense Trend Analysis Results

The category breakdown analysis correctly identified the highest-spending categories from sample datasets supplied during testing. In a test dataset of sixty expense records distributed across five categories (Purchase, Utilities, Maintenance, Salary, and Transport), the module correctly computed the total and percentage share for each category. The resulting pie chart rendered

accurately on the Reports Screen, with category labels and percentage values matching the manually computed expected values in all cases.

The month-on-month bar chart rendered correctly for datasets spanning between three and twelve months. Months with no expense records were correctly represented as zero-value bars rather than being omitted from the chart, preserving the continuity of the time axis. The percentage change figure displayed above the chart correctly indicated positive or negative trend direction in all tested cases.

7.4.2 Forecasting Accuracy

The linear regression forecasting model was evaluated against three synthetic test datasets representing different spending patterns: a steadily increasing trend, a flat stable trend, and a volatile irregular trend.

Table 7.1 Forecasting Accuracy

Dataset	Pattern	Actual Month N+1	Forecasted Value	Absolute Error	% Error
Dataset A	Steadily increasing (PKR 10k/month growth)	PKR 82,000	PKR 80,500	PKR 1,500	1.8%
Dataset B	Flat / stable (PKR 50k each month)	PKR 50,000	PKR 50,000	PKR 0	0.0%
Dataset C	Volatile / irregular (range PKR 20k–90k)	PKR 47,000	PKR 58,200	PKR 11,200	23.8%

The results confirm that the linear regression model performs excellently for stable and steadily trending data, achieving errors below 2% in both cases. For volatile data with no clear linear pattern, the model's error rises substantially (23.8%), which is an expected limitation of linear regression applied to noisy

time series. This finding is consistent with established literature on short-horizon forecasting, where simple regression is well-suited to stable patterns but is outperformed by more sophisticated models such as ARIMA or exponential smoothing when dealing with irregular data (Hyndman & Athanasopoulos, 2021).

For the target user group of small businesses with relatively predictable recurring expenses, the model's performance is considered adequate. The recommendation text accompanying the forecast on the Reports Screen advises users to treat the figure as a planning estimate rather than a precise prediction, managing user expectations appropriately.

7.4.3 Low-Stock Ranking Results

The low-stock restocking list was evaluated with a dataset of twenty inventory items, five of which had quantities at or below their individual thresholds. The module correctly identified all five qualifying items, ranked them in ascending order of remaining quantity, and presented them as a prioritised list on the Predictions tab of the Reports Screen. The ranking correctly placed items with zero remaining stock above items with one or two units remaining, providing an actionable priority order for restocking decisions.

7.4.4 Auto-Generated Recommendation Text

The recommendation text generated by the module was evaluated qualitatively by the five UAT participants. Three participants rated the recommendations as clear and actionable. One participant noted that the phrasing was sometimes repetitive across sessions when spending patterns had not changed significantly. One participant suggested that the recommendations would be more useful if they included specific product names from the inventory in the restocking advice rather than only the category. These observations are noted as directions for improving the natural language generation component in future iterations.

7.5 User Evaluation Results

The user acceptance testing results presented in Chapter 6 are discussed here in terms of their implications for the system's design decisions and the project's contribution to the problem domain.

The average ease-of-use score of 4.6 out of 5 indicates that the interface design goals of simplicity, consistency, and intuitive navigation were successfully realised. The bottom navigation bar structure, the card-based list design, and the confirmation dialogs for destructive actions were each cited positively by at least two participants during the qualitative feedback session. These findings validate the design principles adopted in Chapter 4.

The average clarity of information score of 4.4 out of 5 for the Reports Screen is particularly significant, as the reports module represents the most complex information presentation in the application. The fact that non-technical business users found the AI-generated charts and figures clear and understandable confirms that the decision to use a visual-first reporting approach with minimal technical jargon was appropriate for the target audience.

The feature requests raised by participants (dark mode and spreadsheet export) indicate that the core functionality is considered complete and satisfactory, and that users' attention has moved toward enhancement preferences rather than fundamental usability concerns. This is a positive indicator of the system's maturity relative to its intended scope.

7.6 Performance Results Summary

The performance testing results from Chapter 6 are summarised and interpreted here. The key performance metrics observed are presented in the following table.

Table 7.2 Performance Results

Metric	Normal Load	Moderate Load	Acceptable Threshold
Screen load time (list screens)	< 1.0 sec	~1.8 sec	< 3.0 sec
Firestore stream update propagation	~300 ms	~400 ms	< 1.0 sec
Report generation time	~1.2 sec	~3.1 sec	< 5.0 sec
Login time (network round trip)	~1.5 sec	~1.5 sec	< 3.0 sec
Application crash count (all tests)	0	0	0

All measured metrics remained within acceptable thresholds under both normal and moderate load conditions. The report generation time of 3.1 seconds under moderate load approaches the upper end of the acceptable range but does not breach it. This result reflects the client-side nature of the analytics computation; as data volumes grow beyond the moderate-load scenario, report generation time would continue to increase. The architectural provision for migrating the analytics to Firebase Cloud Functions, discussed in Chapter 4, provides a clear upgrade path should performance become a concern for higher-volume users in the future.

The absence of any application crashes across all test scenarios is a strong reliability result, reflecting the robustness of the error handling implemented throughout the service and UI layers.

7.7 Comparative Discussion

The literature review in Chapter 2 identified several gaps in existing inventory and expense management systems relevant to small businesses. This section discusses how the results of this project address those gaps.

7.7.1 Affordability and Accessibility

Existing commercial systems such as QuickBooks, Zoho Inventory, and TradeGecko were identified in the literature as too expensive or too complex for micro and small businesses in developing economies. The system developed in this project uses Firebase's free-tier services (Spark plan), which allows deployment at zero recurring infrastructure cost for the expected data volumes of a small business. The Flutter-based Android application requires no subscription fee, addressing the affordability gap directly. UAT results confirm that non-technical users can operate the system without training, addressing the complexity gap.

7.7.2 AI-Driven Insights for Non-Technical Users

The literature identified a gap between the availability of AI-powered analytics in enterprise systems and the absence of such features in tools accessible to small businesses (Morabito, 2015; Wamba et al., 2017). The UAT results confirm that non-technical users can derive meaningful insights from the system's AI reports, validating the practical utility of this approach.

7.7.3 Real-Time Data Synchronization

Studies reviewed in Chapter 2 noted that many small business tools require manual data export and re-import workflows, creating delays and opportunities for data inconsistency. The Firestore stream-based architecture of this system eliminates such delays, propagating changes to all connected views within approximately 300 milliseconds. This represents a meaningful improvement over traditional file-based or batch-synchronisation approaches used in many small-business tools.

7.7.4 Limitations Relative to Enterprise Systems

It is important to acknowledge that the system developed in this project does not match the full feature set of enterprise inventory systems. It does not support barcode scanning, multi-location inventory, purchase order generation, supplier management, or integration with accounting software such as FreshBooks or Xero. The system is intended as a practical and accessible foundation for businesses that currently use no digital system at all, not as a replacement for a full enterprise resource planning (ERP) suite.

7.8 Discussion of Key Findings

Several cross-cutting findings emerge from the results presented in this chapter that are worth highlighting collectively.

First, the decision to implement all analytics client-side in Dart was validated by the performance results. For the data volumes typical of a small business, client-side computation is fast enough to be imperceptible to the user in normal use, and avoids the latency, cost, and complexity of server-side processing. This finding supports the argument that lightweight AI features can be embedded directly in mobile applications without a cloud-compute backend, at least at small scales.

Second, the Agile development approach proved effective for a project of this scope. The iterative sprint structure allowed the five defects identified during testing to be discovered and resolved within the same or the following sprint, rather than accumulating into a large remediation effort at the end of the project. The integration test failure on Defect D-01 (the low-stock threshold comparison error) is a clear example of a defect that would have been much more costly to find and fix if testing had been deferred to the final phase.

Third, the UAT results confirm that there is genuine user demand for AI-driven insights among small business owners who currently have no access to such features. The fact that four out of five participants described the forecasting and restocking recommendations as features they would use in their real business

operations is strong evidence of the system's practical value beyond its academic context.

Fourth, the security testing results demonstrate that Firebase Security Rules, when correctly written and tested, provide a robust and practical access control layer for mobile applications. The server-side enforcement of rules means that the security posture does not depend solely on the client application behaving correctly, which is a sound security design principle.

7.9 Summary

This chapter presented and discussed the results of the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators across six dimensions: objective fulfilment, system functionality, AI analytics outputs and accuracy, user evaluation, performance, and comparative analysis.

All four primary project objectives were achieved, with the AI analytics objective qualified as substantially achieved due to the known limitations of linear regression under volatile data conditions. All twelve functional requirements were confirmed as met. The AI analytics module delivered correct trend analysis and low-stock ranking in all test cases, and achieved forecasting errors below 2% for stable and trending datasets. User acceptance testing returned average satisfaction scores above 4.4 out of 5 across all three dimensions, with qualitative feedback confirming the system's practical utility. Performance remained within acceptable thresholds under both normal and moderate load. The system addresses the key gaps identified in the literature review: it is affordable, accessible to non-technical users, delivers real-time data synchronisation, and makes AI-driven business insights available at a scale previously limited to enterprise tools.

The following and final chapter presents the conclusion of the project, reflects on lessons learned, and outlines directions for future work that could extend the system's capabilities.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Introduction

This final chapter brings together the key outcomes of the project, reflects on the journey from problem identification to a working system, and outlines the directions in which this work could be extended. Chapter 7 presented and discussed the results of the system in detail; this chapter synthesises those findings into a cohesive conclusion and situates the project's contribution within the broader context of technology for small businesses.

The chapter is structured as follows. Section 8.2 provides a concise summary of the entire project. Section 8.3 presents the conclusion, evaluating the extent to which the project achieved its aims. Section 8.4 reflects on the lessons learned during the project. Section 8.5 discusses the limitations of the current system. Section 8.6 outlines directions for future work. Section 8.7 presents closing remarks.

8.2 Project Summary

This project is set out to design, develop, and evaluate a mobile application — the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators — to address the inventory and financial management challenges faced by small businesses that currently rely on manual processes such as paper records or basic spreadsheets. The motivating observation was that while powerful inventory and analytics tools exist, they are typically priced, sized, and technically complex in ways that put them out of reach for micro and small enterprises, particularly in developing economies.

A comprehensive literature review in Chapter 2 established the theoretical and practical context, identifying key gaps in existing systems: lack of affordability, poor usability for non-technical users, absence of AI-driven insights at the small-business scale, and limited real-time data accessibility. Chapter 3 translated these gaps into a structured set of functional and non-functional requirements and defined the system architecture. Chapter 4 produced detailed design artefacts including UI wireframes, database schemas, class

diagrams, an entity-relationship model, and the AI analytics component design. Chapter 5 described the implementation of the system in Flutter and Firebase across five modules: authentication, inventory management, expense tracking, AI analytics, and admin. Chapter 6 presented a multi-level testing programme covering unit, integration, system, user acceptance, performance, and security testing, identifying and resolving five defects. Chapter 7 evaluated the results, confirming the fulfilment of all four primary objectives and all twelve functional requirements, and placing the system's outcomes in the context of existing literature.

The completed system is a functional Android mobile application that allows small business owners to manage their inventory and expenses in real time, receive intelligent reports including expense forecasting and low-stock alerts, and administer user access — all through an intuitive interface that requires no technical training to operate.

8.3 Conclusion

The central research question addressed by this project was: can a lightweight, affordable, and AI-assisted mobile application meaningfully improve inventory and expense management for small businesses that currently have no access to digital management tools? The results of this project provide a positive answer to that question.

The system successfully demonstrates that core AI capabilities — specifically statistical trend analysis and linear regression-based forecasting — can be delivered within a mobile application without requiring server-side machine learning infrastructure, cloud compute costs, or data science expertise from the user. The user acceptance testing results, with average satisfaction scores of 4.4 to 4.6 out of 5, confirm that the system is not only technically functional but genuinely usable and valued by members of the target audience. Four out of five participants in user acceptance testing indicated that the AI reports

features were ones they would use in their actual business operations, validating the practical relevance of the project beyond its academic context.

The Firebase backend provided a robust, scalable, and zero-cost-at-prototype-scale foundation that is well-matched to the project's requirements. Real-time Firestore stream synchronisation eliminated the manual refresh and data export workflows that characterise many existing small-business tools. Server-side Firestore Security Rules provided a strong data access control layer that was confirmed effective through direct security testing, giving the system a security posture comparable to commercial alternatives.

The project makes several concrete contributions. At the technical level, it demonstrates a replicable architecture for building lightweight AI-assisted mobile business applications using Flutter and Firebase. At the practical level, it delivers a working prototype that could be deployed and used by real small businesses with minimal additional development. At the academic level, it contributes an evaluation of linear regression forecasting in a real-world small-business data context, with honest documentation of both its strengths and its limitations.

In conclusion, the Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators achieves its stated objectives, addresses the gaps identified in the literature, and demonstrates that AI-powered business intelligence is not exclusively the domain of large enterprises with substantial technology budgets. With the right architectural choices, meaningful intelligence can be delivered to any business owner through an affordable, accessible mobile application.

8.4 Lessons Learned

The development of this project generated a number of technical and process-level insights that are worth documenting for the benefit of future similar projects.

8.4.1 Agile Development for Academic Projects

The Agile sprint structure proved highly effective for a project of this scope. Breaking the development into focused two-week sprints with a defined deliverable per sprint prevented the accumulation of technical debt and ensured that testing was integrated throughout development rather than deferred to the end. The five defects identified during testing were all discovered and resolved within the same or the immediately following sprint, which would not have been possible under a traditional waterfall approach where testing is a terminal phase. For future academic software projects of similar scale, an Agile approach with sprint-level testing is strongly recommended.

8.4.2 Firebase as a Rapid Development Platform

Firebase significantly accelerated the backend development of this project. Authentication, real-time database access, and security rules were all operational within the first development sprint, allowing subsequent sprints to focus entirely on application features rather than backend infrastructure. The main technical challenge encountered was the Firestore limitation on cross-field queries, which required a client-side filtering workaround. This limitation is well-documented but is easy to overlook during the design phase; future projects using Firestore should explicitly audit their query requirements against Firestore's query capabilities during the design phase rather than discovering the constraint during implementation.

8.4.3 Importance of User Involvement

The user acceptance testing phase yielded insights that would not have been apparent from developer-only testing. In particular, the feedback regarding the repetitive phrasing of recommendation text and the desire for product-specific advice in the restocking list were observations that only emerged when real users interacted with the system in a natural way. Involving target users earlier in the design process — for example, through paper prototype reviews at the design stage — would likely have surfaced some of these observations sooner

and informed the design of the recommendation text generator before implementation.

8.4.4 Managing Scope Effectively

A recurring challenge throughout the project was managing the temptation to add features beyond the defined scope. Several candidate features — including barcode scanning, CSV export, multi-currency support, and push notifications for low-stock alerts — were identified and deferred to future work. Maintaining a strict scope boundary allowed the core system to be completed, tested, and evaluated to a high standard within the project timeline. The future work section below revisits these deferred features as organised and prioritised directions rather than abandoned ideas.

8.5 Limitations

While the system achieves its stated objectives, it is important to acknowledge its limitations honestly, as these define the boundaries of the project's contribution and the starting points for future work.

The first limitation is the forecasting model's sensitivity to data volatility. As demonstrated in the results chapter, the linear regression model achieves low error rates for stable and steadily trending data but degrades significantly (approximately 24% error) for volatile, irregular spending patterns. Small businesses with highly seasonal or unpredictable expense profiles may find the forecast less reliable. More sophisticated time series models would reduce this limitation but were beyond the scope of this project.

The second limitation is the absence of offline functionality. The system requires an active internet connection for all Firestore read and write operations. If connectivity is lost, users cannot add, view, or update records until the connection is restored. Flutter and Firebase do offer offline persistence capabilities through Firestore's local cache, but this feature was not implemented in the current version, as it introduced additional complexity around conflict resolution that was outside the project scope.

The third limitation is the platform scope. The system was developed and tested exclusively on Android. While Flutter supports cross-platform compilation to iOS from the same codebase, the application has not been tested on iOS devices, and iOS-specific configuration (such as Apple developer account setup and iOS Firebase integration) was not performed. The system cannot therefore be considered a validated iOS application at this stage.

The fourth limitation is the simplicity of the recommendation text. The auto-generated recommendations are based on fixed templates populated with computed values. They are informative but not adaptive; they do not change their phrasing or depth of insight based on the severity of the trend, the user's history of acting on previous recommendations, or contextual factors such as the business sector. A more sophisticated natural language generation approach would produce richer and more personalized insights.

The fifth limitation is the absence of multi-user collaboration features. The system supports multiple independent user accounts, each with their own data, but does not support shared data spaces where, for example, a shop owner and a store manager could both access and update the same inventory from separate devices. This is a common requirement in small businesses with more than one staff member involved in stock management.

8.6 Future Work

The limitations identified above, combined with the feature suggestions raised during user acceptance testing, define a clear and prioritized roadmap for extending the system. The following subsections describe the most significant future development directions.

8.6.1 Advanced Forecasting Models

Replacing or supplementing the linear regression model with more sophisticated time series forecasting algorithms is the highest-priority technical enhancement. Exponential smoothing (particularly Holt-Winters triple exponential smoothing) would better handle seasonality and trend changes,

making forecasts more reliable for businesses with seasonal expense patterns. An ARIMA model would improve accuracy for irregular data. These models could be implemented client-side in Dart for small datasets or migrated to a Python-based Firebase Cloud Function for larger ones. Providing users with a model selection option based on their data characteristics would further improve the system's adaptability.

8.6.2 Barcode Scanning

Barcode scanning would significantly accelerate inventory data entry, particularly for businesses that stock packaged goods with standard barcodes. Flutter's mobile_scanner package provides camera-based barcode and QR code scanning capabilities that could be integrated into the Add/Edit Item form, allowing users to scan a product barcode and auto-populate the item name and category from a product database API such as Open Food Facts or Barcode Lookup. This feature was consistently mentioned as a desirable enhancement during informal discussions with potential users outside the formal UAT session. Although it was meant to be a future feature, I decided to add it anyway in my current project.

8.6.3 Data Export

The ability to export inventory and expense data as a CSV or PDF file was requested by one UAT participant and is a standard expectation in business software. Flutter's path_provider and csv packages, combined with the pdf package, would enable the generation of downloadable reports that users could share via email or store for accounting purposes. An export function on the Reports Screen producing a monthly expense summary PDF would be particularly valuable for users who need to share financial records with an accountant or tax authority.

8.6.4 Offline Support

Enabling Firestore's local persistence would allow the application to function without a network connection, queuing writes locally and synchronizing them with Firestore when connectivity is restored. This enhancement would make

the system viable in environments with unreliable internet access, which is a relevant constraint for many small businesses in developing economies. Conflict resolution logic would need to be implemented carefully to handle scenarios where the same record is edited offline on multiple devices simultaneously.

8.6.5 Push Notifications for Low-Stock Alerts

Currently, low-stock alerts are displayed only when the user is actively viewing the Inventory List Screen or the Reports Screen. Implementing push notifications using Firebase Cloud Messaging (FCM) would allow the system to alert the user proactively when an item's quantity falls below its threshold, even when the application is closed. A Firebase Cloud Function triggered by Firestore document writes could evaluate the updated quantity against the threshold and dispatch an FCM notification to the user's device if the threshold is breached.

8.6.6 Multi-User Collaboration

Supporting shared inventory spaces would make the system suitable for businesses where multiple staff members need to manage the same stock. This would require a significant data model extension: introducing a Business entity that owns Inventory and Expense collections, and modifying the Users collection to support membership of one or more businesses with role assignments per business. Firebase's existing authentication and security rules infrastructure provides a solid foundation on which this multi-tenancy model could be built.

8.6.7 iOS Support and App Store Deployment

Completing the iOS build configuration and conducting device testing on iOS hardware would allow the application to be submitted to the Apple App Store, substantially broadening its potential user base. Since the application is already built in Flutter, the iOS-specific work is primarily configuration rather than code rewriting, making this a relatively low-effort extension relative to the value it delivers.

8.6.8 Machine Learning-Driven Category Suggestions

A longer-term enhancement would be to use on-device machine learning to suggest expense categories automatically based on the title entered by the user. A simple text classification model trained on a labelled dataset of expense descriptions could be embedded in the application using TensorFlow Lite, reducing the manual effort required for category assignment and improving data consistency across users.

8.7 Closing Remarks

This project began with a simple but important observation: that the tools most needed by the businesses with the fewest resources are often the ones least likely to be available to them. Small business owners managing inventory with notebooks and expenses in spreadsheets are not lacking in ambition or capability; they are lacking accessible tools that meet them where they are.

The Smart Inventory and Expense Tracker with AI Reports For Key Performance Indicators is a step toward closing that gap. It is not the final answer — no single project could be — but it demonstrates that the gap is closeable with current technology, thoughtful design, and a genuine commitment to building for the user rather than building to impress. The satisfaction scores and qualitative feedback from user acceptance testing suggest that when software is designed with a specific user in mind, those users notice and respond.

Flutter and Firebase proved to be an effective and productive technology stack for this class of application, and the architectural patterns established in this project — particularly the service-layer abstraction, client-side analytics with a clear migration path to cloud functions, and Firestore Security Rules for access control — provide a reusable template for future applications in this domain.

The author hopes that the system, the architectural decisions, the testing methodology, and the honest evaluation of results documented in this thesis will be useful both to practitioners building similar tools and to researchers

studying the application of lightweight AI in resource-constrained contexts. The work is not finished — the future work section is evidence of that — but it is a solid and well-evaluated beginning.

REFERENCES

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. https://otexts.com/fpp3/

Morabito, V. (2015). Big data and analytics: Strategic and organizational impacts. Springer.

Wamba, S. F., Gunasekaran, A., Akter, S., Ren, S. J., Dubey, R., & Childe, S. J. (2017). Big data analytics and firm performance: Effects of dynamic capabilities. Journal of Business Research, 70, 356–365. https://doi.org/10.1016/j.jbusres.2016.08.009

Document Viewer

Turnitin Originality Report

Processed on: 23-Jul-2026 11:36 PKT
ID: 3002129000
Word Count: 21459
Submitted: 1

Usman By Muhammad Abid

Similarity Index	Similarity by Source	
10%	Internet Sources:	6%
	Publications:	3%
	Student Papers:	7%

exclude quoted | exclude bibliography | exclude small matches | mode:
quickview (classic) report | print | refresh | download

1% match (student papers from 26-Oct-2010)
Submitted to Higher Education Commission Pakistan on 2010-10-26

<1% match (student papers from 20-Jul-2026)
Submitted to Higher Education Commission Pakistan on 2026-07-20

<1% match (student papers from 04-Sep-2009)
Submitted to Higher Education Commission Pakistan on 2009-09-04

<1% match (student papers from 14-Mar-2012)
Submitted to Higher Education Commission Pakistan on 2012-03-14

<1% match (student papers from 31-Oct-2018)
Submitted to Higher Education Commission Pakistan on 2018-10-31

<1% match (student papers from 06-Mar-2026)
Submitted to University of Wales Institute, Cardiff on 2026-03-06

<1% match (student papers from 09-Mar-2026)
Submitted to University of Wales Institute, Cardiff on 2026-03-09

<1% match (student papers from 10-Mar-2026)
Submitted to University of Wales Institute, Cardiff on 2026-03-10

<1% match (student papers from 01-May-2026)
Submitted to University of Wales Institute, Cardiff on 2026-05-01

<1% match (student papers from 15-May-2026)
Submitted to Arab Open University on 2026-05-15

<1% match (student papers from 05-May-2025)
Submitted to Arab Open University on 2025-05-05

<1% match (student papers from 14-May-2026)
Submitted to Arab Open University on 2026-05-14